Transaction Concepts (follow the book)

What is Transaction?

- you may hear a term very often at work and people refer to many meaning, nowadays we have standard meaning

S

S #	SNAME	STATUS	CITY
S 1	SMITH	20	LONDON
S 2	JONES	10	PARIS
S 3	BLAKE	30	PARIS
S 4	CLARK	20	LONDON
S 5	ADAMS	30	ATHENS

SP

S #	P #	QTY
S 1	P 1	300
S 1	P 2	200
S 1	P 3	400
S 1	P 4	200
S 1	P 5	100
S 1	P 6	100
S 2	P 1	300
S 2	P 2	400
S 3	P 2	200
S 4	P 2	200
S 4	P 4	300
S 4	P 5	400

P

P #	PNAME	COLOR	WEIGHT	CITY
P 1	Nut	Red	12	London
P 2	Bolt	Green	17	Paris
P 3	Screw	Blue	17	Rome
P 4	Screw	Red	14	London
P 5	Cam	Blue	12	Paris
P 6	Cog	Red	19	London

The supplier-and-parts database

- S table represent object instances or entity instances of supplier.(master table)
- P represent object instances or entity instances of part. (master table)
- SP represent relationship instance between supplier and part. (transaction table) —

> The term “transaction” has several meanings, depend on the popularity at the time.

- 🌱 A transaction could mean a table which keeps relationship instances (such as the SP table).
  - Each row is called 1 transactions (surprisingly). —> not actually correct (1970, 1980)

- 🌱 A transaction could refer to a file which keeps changes (master file, transaction file).
  - Historical part of transaction
  - 📍 (1960s) → Master file, and because the tape must be processed sequentially → sorted by a key → so you can search for a key, or else you have to search the entire tape.
    - How to modified this one? (physically at that time it implement on tape.) (sequential sorted by the key)
      - How you insert the data? (insert new records?)
      - How to delete old record?
      - How to modify existing record?
        - Answer: We can't just directly modify the data, we can't do that, we have to keep the changes somewhere → Transaction file(Tx)
          - Tx file → A file which keeps changes to be made to the master file.
        - Example, you have a task master file, inventory master file sorted and you would like to insert, update, delete the inventory → you have to collected those changes into this file (transaction file). **The record must be sorted by the same key as master file.**

          > The idea is you collect all changes and sort by the same key as master file, and periodically which maybe the end of the day, every hour, depends → you open Master file(old master) and transaction file

          Sequential file processing → you open both file, then compare a key, if master file key is less than in value to transaction file key, it means you don't have to do anything → you just write new record to the new master file. (old master, Tx, new master)
          - if the key match, it could be a modify or a delete.
            - if you want to delete some row → you record the key, the value of the key of the row to be deleted into Tx, and when it matches, you simply not write the old record to the new one.
            - If it modify → you write a new value to the new master file
            - insert → the key must be different from the one in the old master

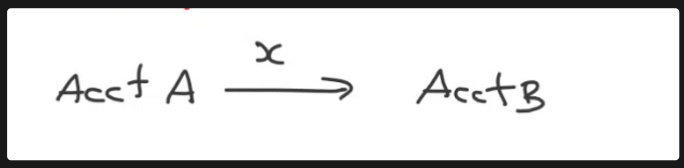
            By checking record, if the key match. key don't match could be insert or error
      - (this very idea is used on blockchain as well, blockchain is sequential)

👉 as conclusion, if you ask people in the 60s, what a transaction is? A transaction is kind of file which keep changes to be made to the master file. (this is transaction in the past)

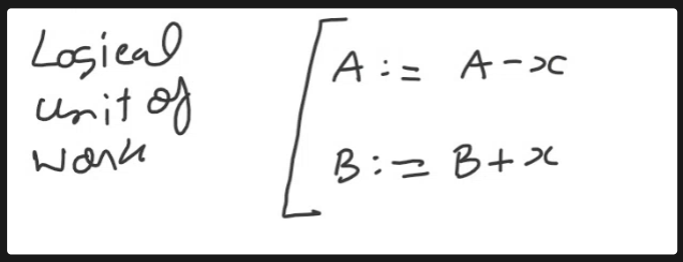
- 📍 1970s-1980s → we have disks, and disk is not sequential like tape → it's direct access.
  - the term transaction is slightly different from the previous one because it direct access, you don't have to accumulate changes into Tx → you can go direct to the place and update the particular record you want to update.
  - 🌱 A transaction is a single database operation.
- What about money transfer? (withdraw from one, deposit to another one) how many transaction for money transfer activity?
  - 1 - withdraw and deposit should go together
  - 2?

nowadays, some people also called single database operation as transaction → not quite correct.

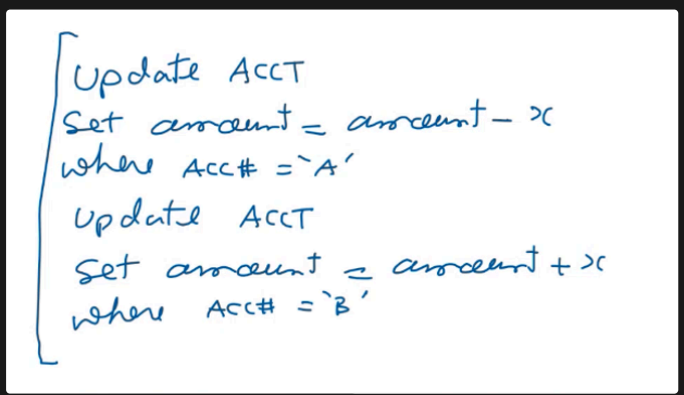
- 📍 1990s → present (definition we used through out lecture in this semester.
  - 🌱 🌱 A transaction is a logical unit of work in which integrity constraints are allowed to be violated. (current day, modern day transaction)
  - What is mean by logical unit of work? (LUW)
    - A logical unit of work is a group (has order) of database operations. (you may have insert, delete, modify operation)
    - for example, transfer of money → you want to move an amount from account A to account B. (transfer from A to B x bath)



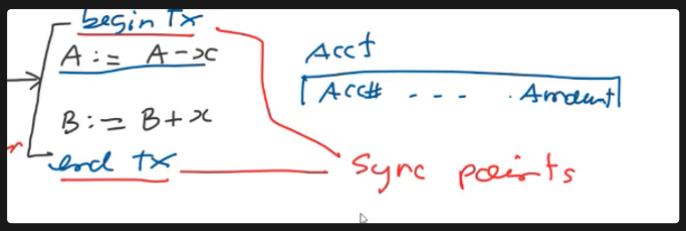
      - in that money transfer activity, in fact, you have withdrawal x bath from A.
      - deposit x bath to b
      - grouping them → Transaction → Logical unit of work.

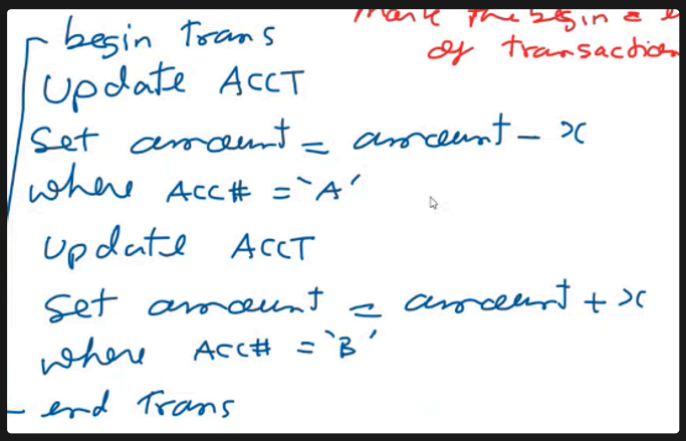


  - Example using SQL



- to mark the begin and the end of transaction
  - Sync points - sync points mark the begin and end of transactions



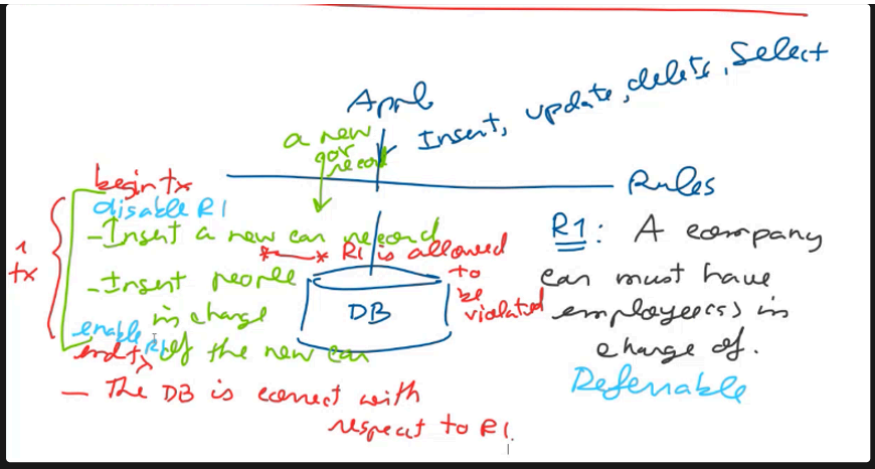


  - the syntax on the sync point depends on language or tool you using.

- What is integrity constraints? why we need concept of integrity constraint here?
  - An integrity constraint or integrity rule or validation rules → are rule that enforce the correctness of the database
    - You may have rule like student may borrow no more than 10 books, you can't enroll in 2 course that have the same exam time.
    - Where should we keep this rule
      - (possible) part of application program, but if the rule change you have to go back to application program and change them. Most people don't want to touch source code unnecessary.
      - nowadays, most integrity constraint are declare on database. And those rule enforce by DBMS.
        - you may declare student can't borrow more than 5 books, and let the DBMS check upon the insertion of borrow record. (if exceed → reject the insertion)

> They should be declare on the database and be enforced by the DBMS → their command like create constraint, create validation rule.

Example



insert, update, delete, select → the operation trigger the rule, if it follow it can be done.

by R1 → you can't have a car without employee take care of it

what if you buy a new car → new car can't be inserted to the system?

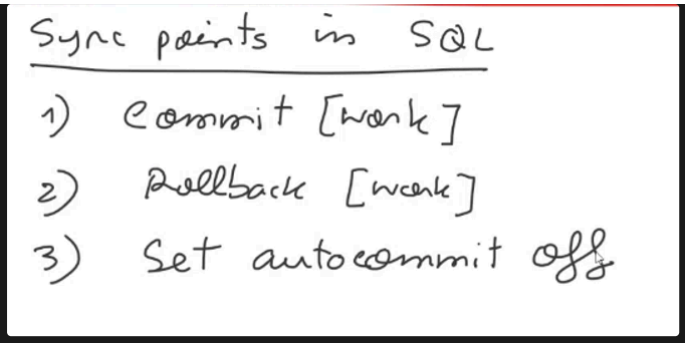
- what should we do to insert a new car?
  - Insert new car record
  - Insert people in charge of the new car
  - (there are 2 operation and we group them together —> we allow R1 to be violated)
- without this concept of transaction - you can't insert a new car.
- the implementation of this concept
  - a very primitive way(manually) → begin with like disable R1, before the end like enable R1.
  - a good and expensive system → you declare rule, you code the rule → put keyword like deferrable. Deferrable mean you don't find the rule inside, after transaction finish, you enforce the rule. R1 is allow to be violated inside, but outside the database is correct with respect to R1.

- Most recent product like Microsoft or oracle → True
- IBM avoid using the term transaction (old company) → They use LUW.

Sync point in SQL

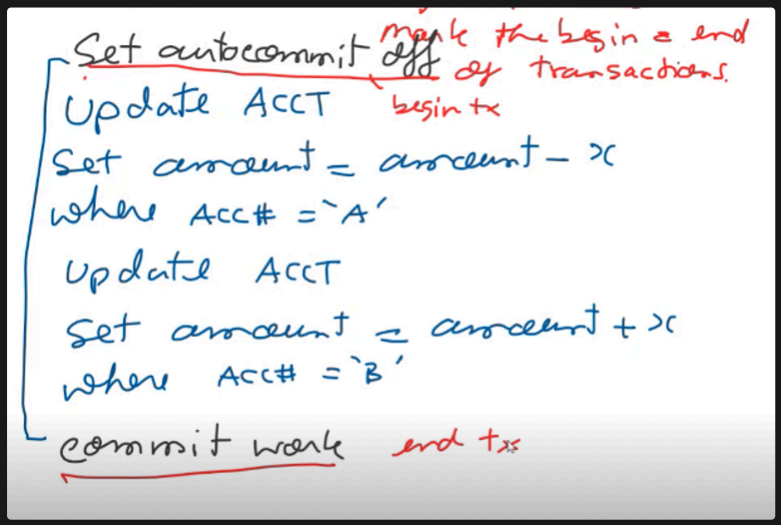
- There are sync point in sql, design to mark the begin and the end of transaction

commit [work (option)]



- Some people (don't know sync point) think commit, rollback is simple sql command → noooo
- commit: database operations from the previous sync point to the commit point are confirmed
  - for example, you doing many activities, and after you commit → the data from commit to previous sync point are confirmed.
- roll back: (opposite of commit) Database operations from the previous sync point to the rollback point are cancelled.
- you can use this commit and roll back command in an interactive mode as well
- set auto commit off (opposite to set auto commit on)
  - The opposite to set auto commit on which forces each operations to commit individually
  - auto commit off - each operation will not be committed. There needs to be an explicit commit operation to confirm the database operations.
  - (what is set auto commit on - after each single sql operation, the operation is committed automatically)

> To do transaction processing, we have to set auto commit off



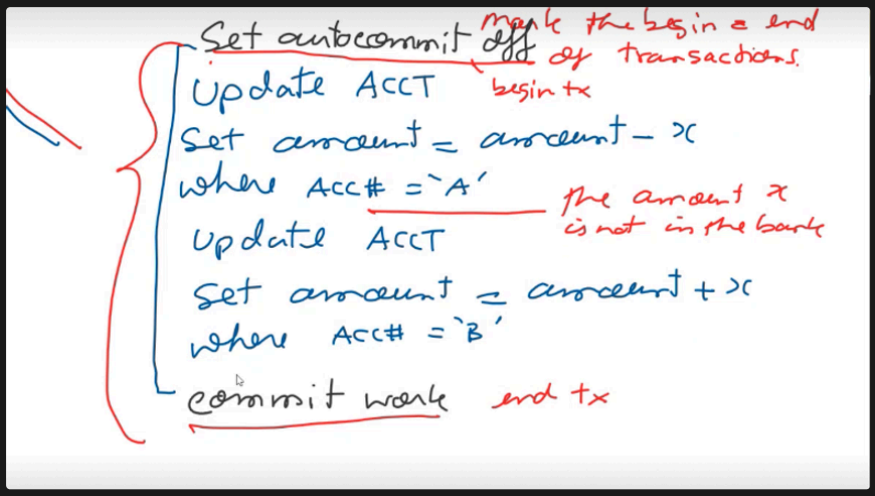
this is a transaction

Instead of using begin transaction and end transaction, we can use set autocommit off, commit

---

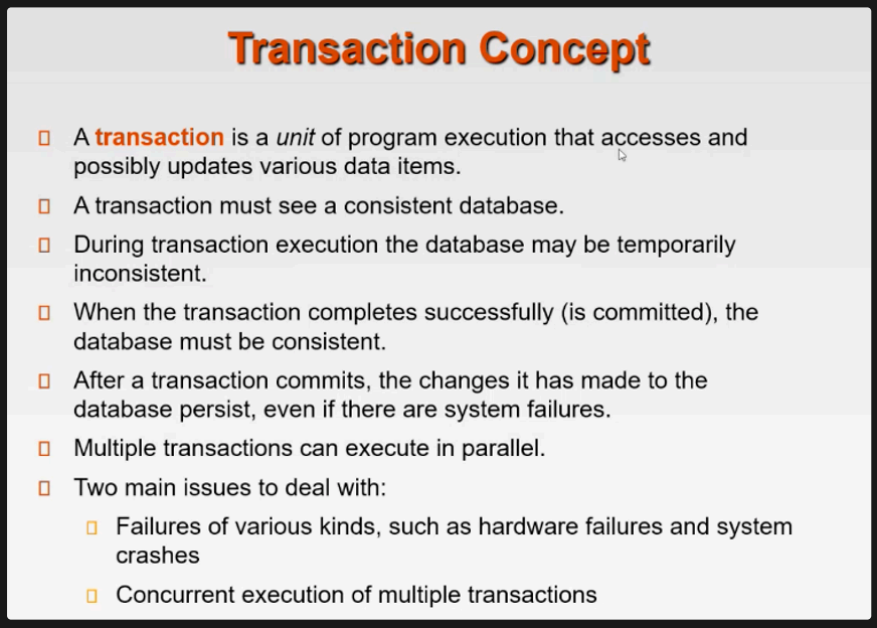
The rule can be explicit or implicitly declare.

- when transfer money
  - implicit → no amount should be lost during transfer.



    - amount x disappear for a short period and arrived back. This a little integrity constraint that allowed to be violated during transaction here.

Definition from a book



read in here too.

A completion of transaction definition must include 4 bullets. (equivalent to our definition, if ask can refer to this)

- A transaction is a unit of program execution that accesses and possibly updates various data items - definition from the book, not yet complete
- A transaction must see a consistent database - it mean that before the transaction, the database is assumed to be correct with respect to integrity constraint.
  - consistent - mean correct with respect to integrity constraint.

- During transaction execution the database may be temporarily inconsistent.
  - during execution of transaction, some rule may be violated, and database become temporarily inconsistent.
- When the transaction completes successfully (is committed), the database must be consistent.

What if there're failure occurs, what does the system do to ensure transaction correctness? (2 main issues)

- How do you know the result is correct.

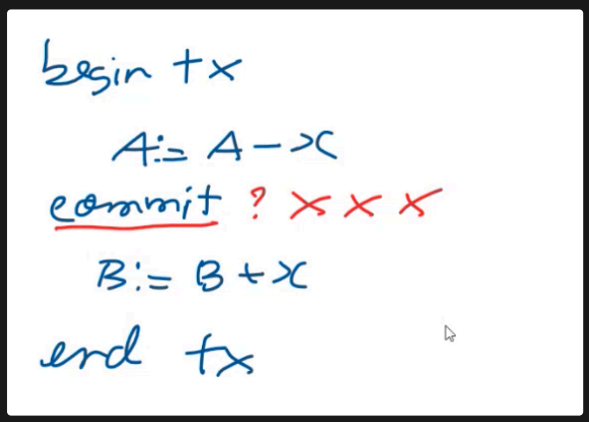
What actually is mean by commit

> 👉 commit: Database operations from the previous sync point to the commit point are confirmed

** it not actually said it write to the disk yet, it just said it confirmed.

Commit is a logical statement, logical level confirmation statement from the DBMS

Some example of wrong



money transfer will be successful if withdraw + deposit both successful

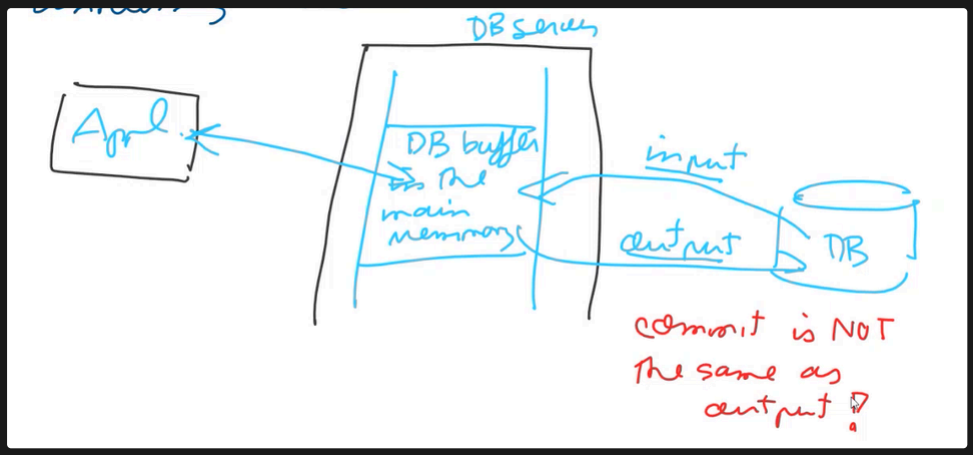
- if withdrawal commit and deposit failed, you can't roll back the whole thing.

some programmer put commit on every after sql statement, why? avoid the lost of data if failure occur → damage the whole idea of transaction.

> 👉 commit does not mean to move the modified data from the main memory to the mass storage.

Example 1: Deposit an amount to a university account. **(commit before output)**

- when our application work with transaction with database, it never work directly with the mass storage (disk). Application always work with the database buffer in a main memory.



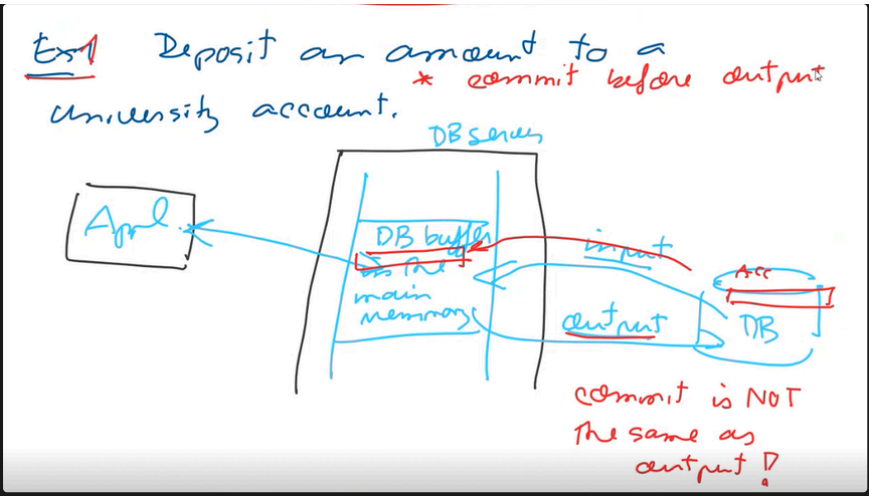
- DB server have database buffer in a main memory, and you have your application, doesn't matter if it run in the same machine or not.
- Application always interact with database buffer, not database on the disk.
  - In the case that your application require any data, the dbms will search inside the DB buffer.
  - if found will return result to your application.
  - if not, it will load data from database space to the buffer (input activity). (load data from mass storage to DB buffer)
  - sometimes, when we no longer use data in the buffer, it can be output back to the space to clear the space and make room for other data. (output)
- from disk to main memory → input
- from main memory to disk → output

> 👉 commit is not the same as output.

people who code incorrectly, their intention is to output.

What if you want to pay some fee to kmitl, and you the first one that want to pay, but the kmitl account is not yet in DB buffer.

So, you want to transfer, DBMS need to input the data to DB buffer. (kmitl acc from disk to DB buffer)



- And your application deposit money to this account
- so it involved input and modified data in main memory (you the first one)
- another student come and pay → transfer to DB buffer
  - after each commit, data modified buffer may not be output to the disk.
  - WHY? because the data is still busy (people still using it), so they no point to outputthe content of the buffer for each and every transaction, the disk will be too busy and slow. The thing that still be in used should be in a main memory, modified them and when finished → dumped to the disk together. NOT EVERY COMMIT MEAN OUTPUT. (maybe to avoid too many disk access)
  - this is case where commit before output.

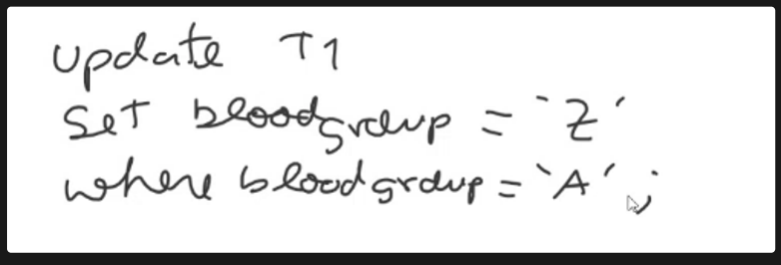
And you can imagine if the system crash and you have few commit in DB buffer → if system crash, you lost main memory. The challenge is how to be recover them, because they committed, so that they will be seen permanent on DB space on the disk → the challenge

Example 2 (interesting point: output before commit)

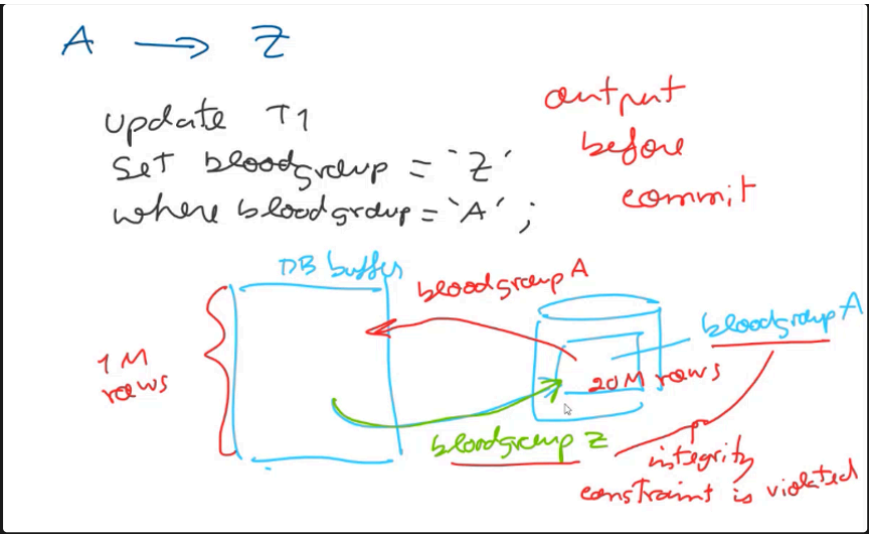
Blood group update.

Suppose we have blood group A, and for some reason WHO said this no longer A, let's called in Z. And imagine you looking through uni database, and there many student with blood group A.

And now you have to update to Z. Operation is easy.



Suppose you looking through Thailand database → 20 m of people have blood group A, and you want to update it to be blood group Z.



- to change from A to Z, DBMS must read 20M row into buffer, and suppose DB buffer is smaller than a disk, and we can fit something like 1M at a time.
  - so, the first 1M to from the disk to main memory, modified A now become Z and what's next? you need to read the next batch, the Z one have to be output back to the disk.
  - On the disk, you have blood group A with Z together.

    > Integrity constraint → you either have blood group Z or blood group A, you should not have A, Z together. but in practice, now we have both Z and A together.

  - Violation of integrity constraint

Output before commit

- You have to force data out even before commit.

Problem is what if the system went down during this operation, you have some Z value output to the disk already → you have A, Z in disk → Challenge, how to remove those uncommitted data like Z out of the DB space.

> Do not mix commit(logical) with output(physical).

case 1: have many commit before output

case 2: have output, modified data to the DB space already before commit.

challenge → **system failure**

case 1: how could we recover those committed transaction, committed already loss in memory

case 2: failed → cancelled

- Two main issues to deal with:
  - Failures of various kinds, such as hardware failures and system crashes
  - Concurrent execution of multiple transactions

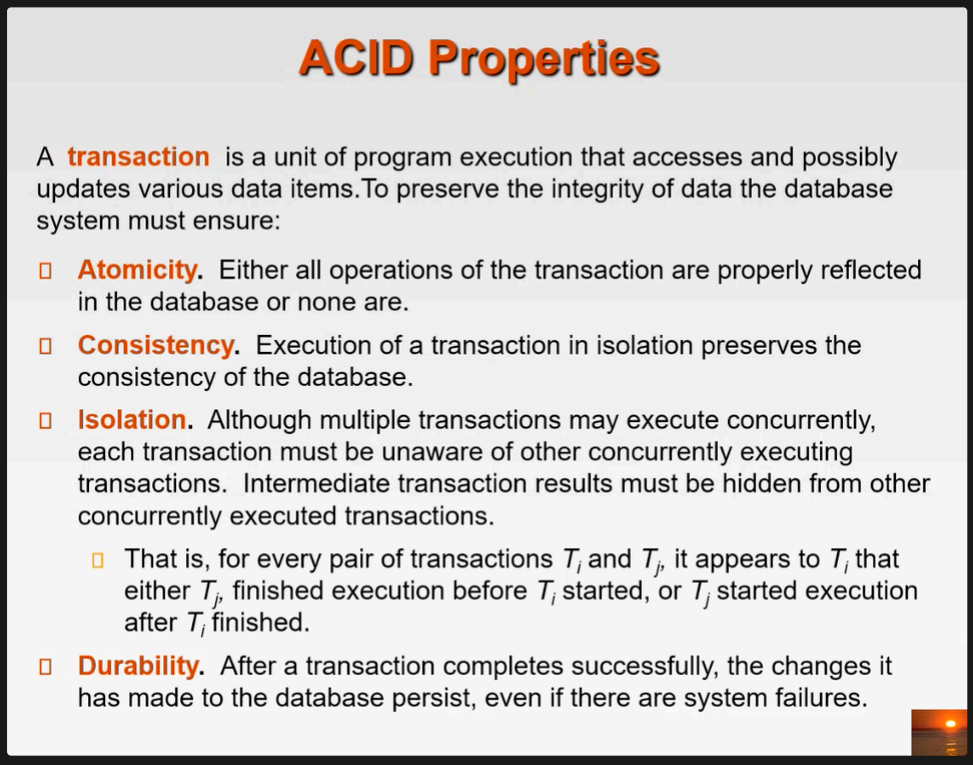
Who will take care of this failure → DBMS not application program (transaction recovery)

Concurrent execution of multiple transactions → you have many transaction running, how can you know the result is correct.

What is correct?

- will discussed in separate chapter

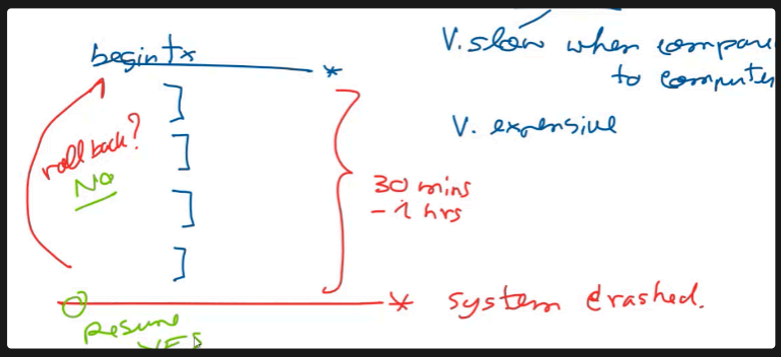
ACID Property (not definition of transaction)



- Good property, recommended property

Some transaction may not have ACID property

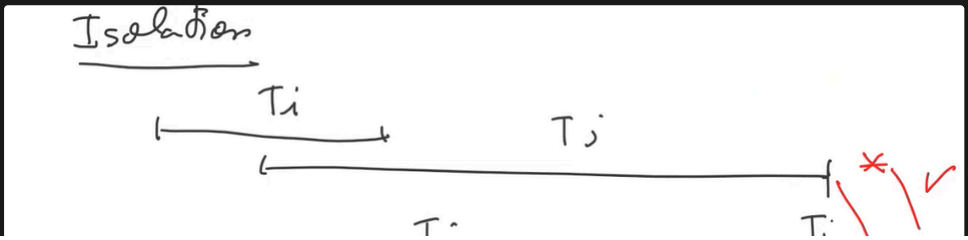
- Atomicity
  - Either all operations are succeeded or all failed
  - Except for long duration transaction
    - What is a long duration transaction?
      - long duration transaction — involve human intervention (case 1, 2 → no human intervention)
      - when we have human intervention, interactive user, interactive user and human are consider slow I/O, we people are very slow compare to computer for example, the access speed could be milisecond, but for us to key something, it could take few second. A screen full could take few minute possible, this is why we call in long duration.
      - Not only very slow, but also very expensive. → cost of human is expensive.
      - So in principle with long duration transaction, the entire transaction should not fail in atomic manner. Instead with a long transaction duration with lot lot of human involvement should failure occur, after failure, the system should rezoom back to the failure point or nearly failure, not to cancelled everything.



      - example, suppose a group of friend would have to have a customized tour → discuss with tour agency to create a tour, tour agency begin transaction → ticket, hotel, etc. → after 30 minutes - 1 hour, system crashed. Do you want whole system to roll back? it took long time to discuss, no one want that. No, we want to keep activity and resume
  - in the transaction, we have many operation like withdraw or deposit.
    - this is consider one unit, either it all failed or it all success. We don't something that half done.

- **Consistency**
  - Execution of a transaction in isolation preserves the consistency of the database.
  - If you have transaction, we allow integrity constraint to be violated inside, and after it back database should be consistent.
  - Exception: in the case of distributed database, suppose you have more than 1 site, you may have duplicate data. Many copy of the same database in many server, many places around the world for example. It possible that we not preserve immediate consistency. Now we have something called **"Eventual consistency",** You update one site, another copy not correct yet, but eventually all with be update and correct by the system one by one. They don't have to be all updated in real time. This help reduce the cost, update and commit one by one.

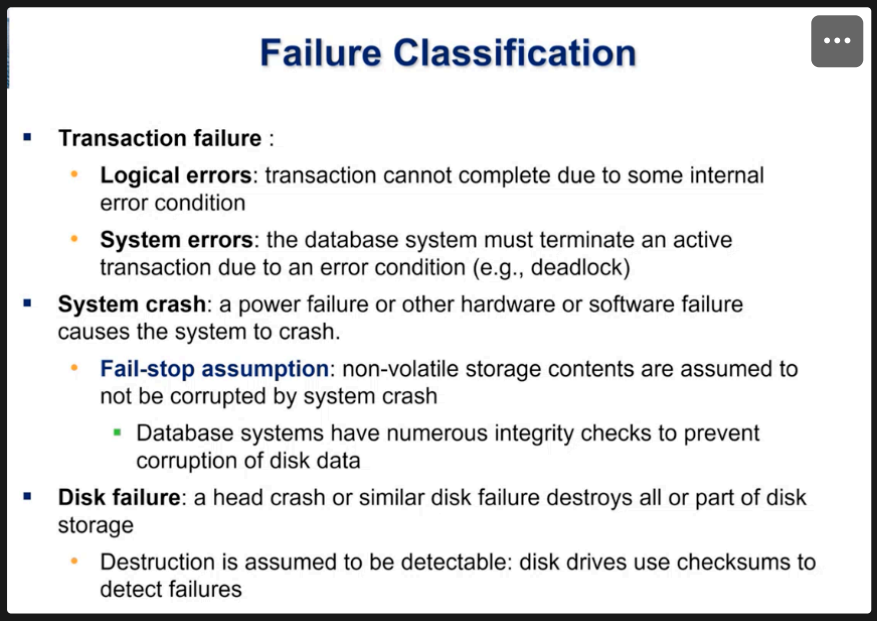
- Isolation
  - **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
    - That is, for every pair of transactions T_i and T_j, it appears to T_i that either T_j, finished execution before T_i started, or T_j started execution after T_i finished.
  - They run separately, they don't see each other.
  - They have many user running, transaction of those user don't see each other.



Transaction Recovery (1)

(Chapter 19 - Recovery System)

Failure Classification



- Transaction failed - only that particular transaction failed, not the entire system.
- System crash - the entire system go down, the computer system crash, but the disk still okay.
- Disk failure - mass storage crash

In this chapter, we mainly deal on transaction failure and system crash → this 2 kind of failure, the dbms will automatically recover transaction for us without using any backup.

- However, for disk failure, backup are required.

🌱 Transaction Failure

Transaction Failure Example

- There was computer center that update lot of data overnight → there are many operation in a transaction, actually, they put 40 database operation in 1 transaction. The transaction deal with million of rows.
- They also keep journal like first statement is finished, second is finish....

- In the morning, transaction failed, on the screen, it said transaction failed → 0 records update, but when they look at the journal → 90 percent of work has been done.

> 👉 What happened? → 0 record update, transaction failed → the whole thing rolled back.

How to prevent this?

- We found that a log file was full (log file keep temporary activity) → log file could not accommodate all those activity.
- Problem → not enough space for log file, too many activity.
- why they put 40 operation in 1 transaction? log file could not accommodate that, transaction failed.
  - they said because it an transaction → they used old definition of transaction (A file which keeps changes to be made to the master file)
  - and there no integrity constraint that they want to violate
    - actually, each of 40 operations can be separate → set auto commit on.
    - became 40 transactions, each commit separately, and because each transaction commit separately → the log file was not full (commit → clear the log file,....)

2 types of cause

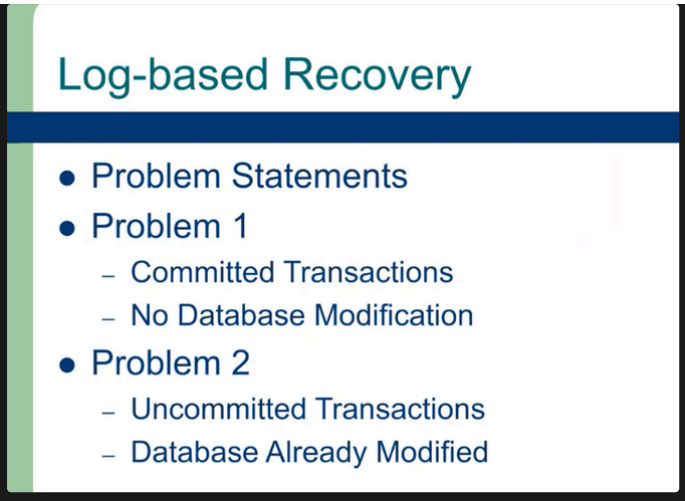
- Logical errors → from poor coding (infinite loop, read bad input, ...)
- System errors → deadlock or log file was full, etc.

🌱 System Crash

- power failure is very common, A power failure or hardware or software failure cause system to crash
- Too many user, don't have enough memory to accommodate them, some system shutdown themselves, possible

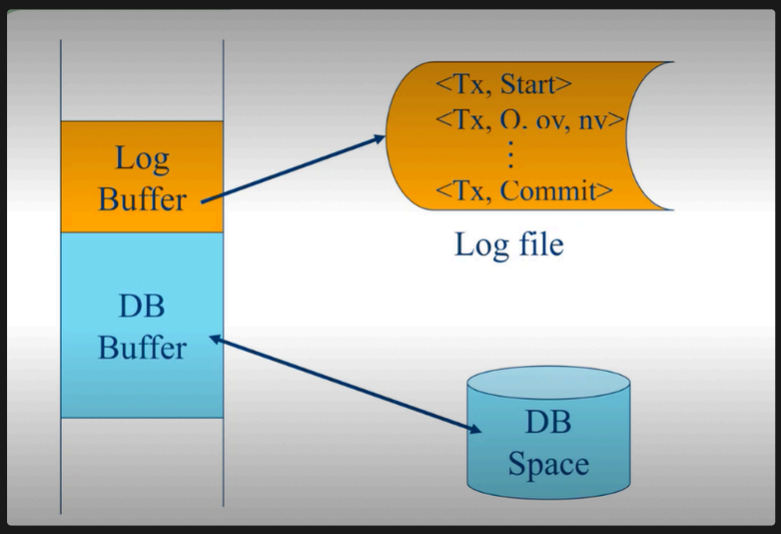
---

How to prevent or solve, what are the problem actually?



- Problem 1 → how to recover those commit transaction that loss in main memory, so that they appear permanent on DB space. (commit before output)
- Problem 2 → you have uncommitted transaction, unfinished, system went down, how to cancel out those(output before commit)(database modification that need to be roll back

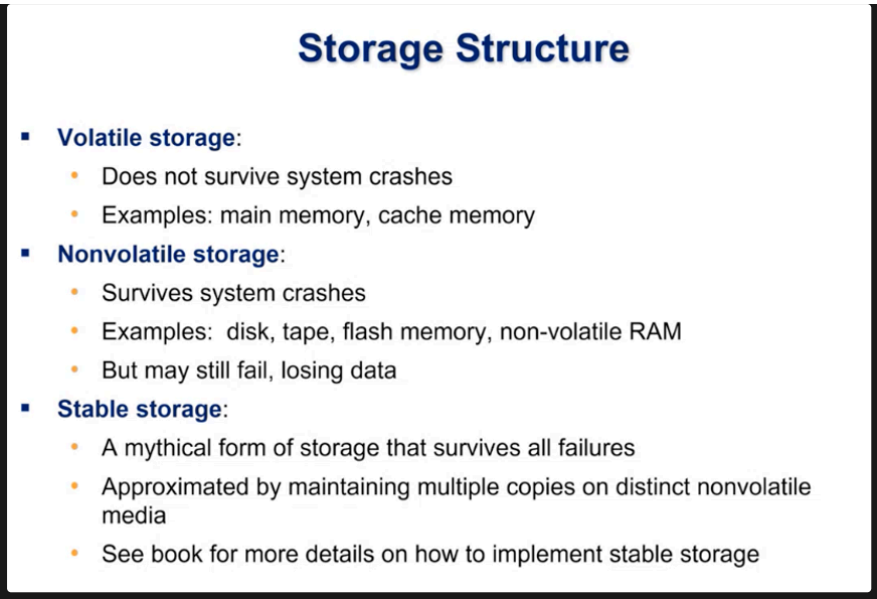
Most popular solution → Log-based Recovery



In principle, we have database buffer and db space.

- We have another buffer call log buffer and log file

Storage Structure

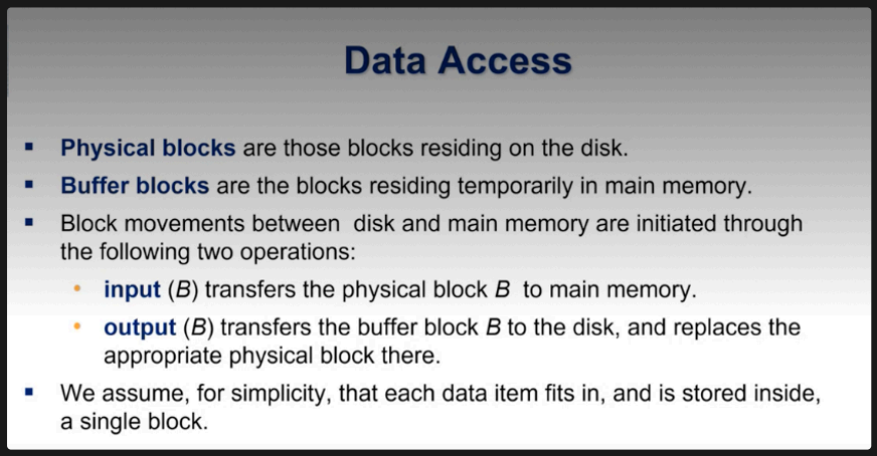


- Stable storage
  - Redundant storage
  - Some DBMS implement stable storage for us, so it can be implemented by software possible

---

Data Access

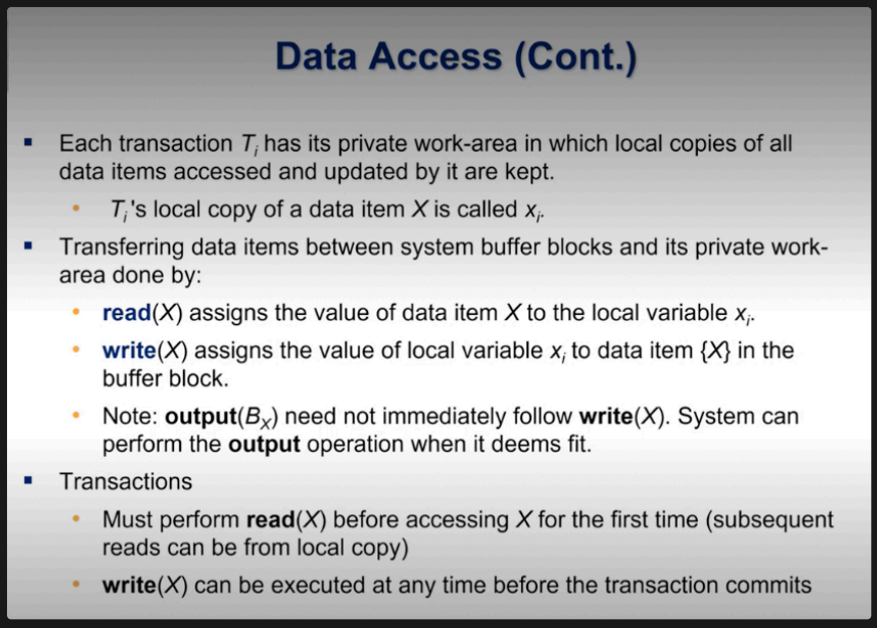
- input and output are transfer between database on mass storage (disk) and the main memory



- when the book or user manual said block → they means database block not OS block
- what's database block
  - smallest unit that transfer from mass storage to main memory.
  - smallest unit the the DBMS read from mass storage.
  - that database block may comprised of many OS block.
- input(B) - transfers the physical block B to main memory.
- output(B) - transfer the buffer block B to the disk.

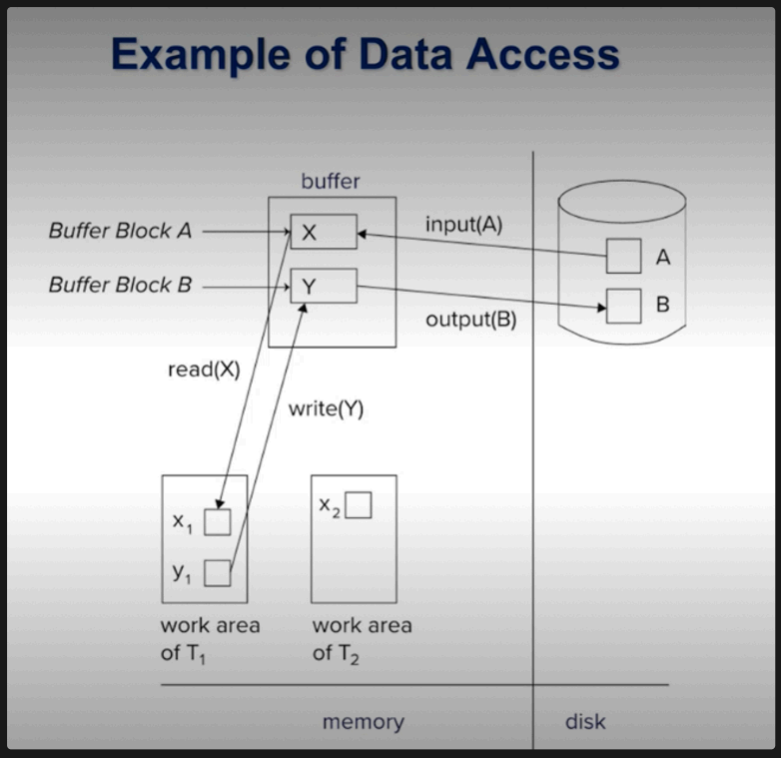
Who control input, output activity?

- **DBMS and the OS**, not our application program.



read in here too, when should it output, when no body else want to use the data or it full and have to get rid of the data first. Output are not control by application program. Input maybe initiate by read or write statement.

- The book refer to the term read and write separately from input, output.



  - Database buffer is in main memory of database server (if you have), part of main memory that keep data for an entire database system.
  - Transaction work area can be in the same machine or also different machine
  - Read → to move data from buffer block to transaction work area
  - write → to write data from transaction work area to database buffer.

> Our program (sql statement) never works with a disk → it work with a buffer.

> Input and output between buffer and DB Space are done by the DBMS and OS.

when your transaction want to read something, and the data already in the buffer → you read it

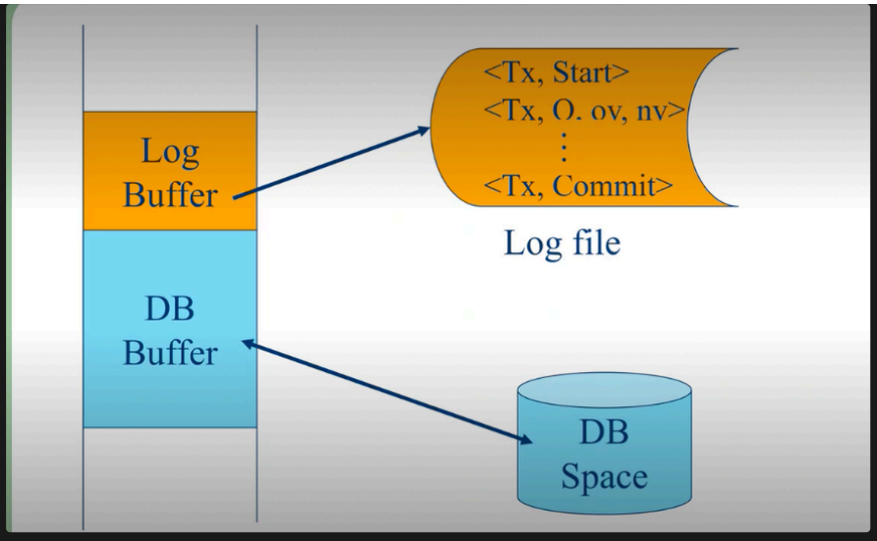
  - but if it not there, system will input the data from disk to buffer (read may initiate input)

say you want to write small y to big Y, but if the big Y is not yet in the buffer → The DBMS need to inout it to the buffer

  - Both read and write may initiate input activity

Log based recovery

  - in here, log file is introduced
  - have separate log buffer to keep log record → those log record will be output to the log file during execution of transaction



During operations, we may input data from db space to the buffer, if the data require by the application, DBMS and OS may input the data. And DBMS and OS may output the data from the buffer to DB Space.

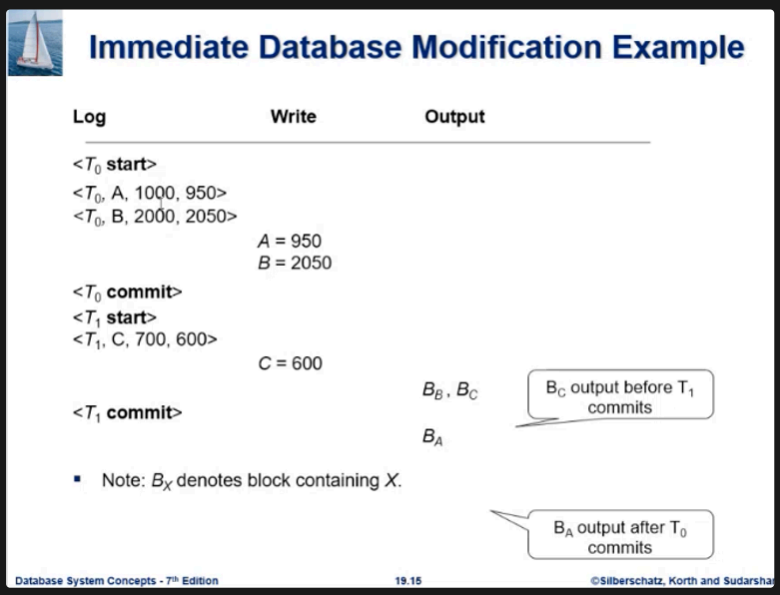
<Tx, Start> → Transaction start (system generate transaction id)

<Tx, Q, ov, nv> → Q is a data item, ov is old value(value before operation, nv → value after operation

we keep on doing this and at the end, we said commit, there are commit record output to the log file. Log file record changes change to database.

- Tx → Transaction identifier

Example of Log record



Immediate DB Modification Recovery Example

Below we show the log as it appears at three instances of time.

(a)	(b)	(c)
<T_0 start>	<T_0 start>	<T_0 start>
<T_0, A, 1000, 950>	<T_0, A, 1000, 950>	<T_0, A, 1000, 950>
<T_0, B, 2000, 2050>	<T_0, B, 2000, 2050>	<T_0, B, 2000, 2050>
	<T_0 commit>	<T_0 commit>
	<T_1 start>	<T_1 start>
	<T_1, C, 700, 600>	<T_1, C, 700, 600>
		<T_1 commit>

Recovery actions in each case above are:

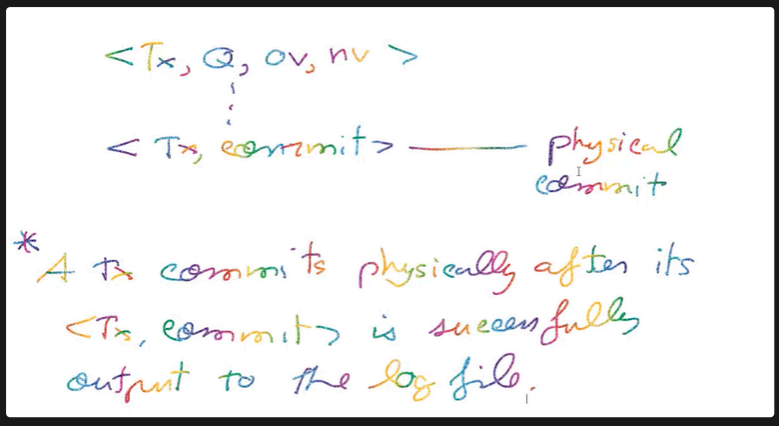
(a) undo (T_0): B is restored to 2000 and A to 1000, and log records <T_0, B, 2000>, <T_0, A, 1000>, <T_0, **abort**> are written out

(b) redo (T_0) and undo (T_1): *A* and *B* are set to 950 and 2050 and C is restored to 700. Log records <T_1, C, 700>, <T_1, **abort**> are written out.

(c) redo (T_0) and redo (T_1): A and B are set to 950 and 2050 respectively. Then *C* is set to 600

- The log record may not come in sequence like this, it doesn't have to be like T0 then T1 → it all mix up, depend on event

- commit record in log file → physical commit



  - if commit already output to the log file → it mean transaction already commit physically.
  - log file == stable storage
  - stable storage → assume not to be loss.

Log based Recovery

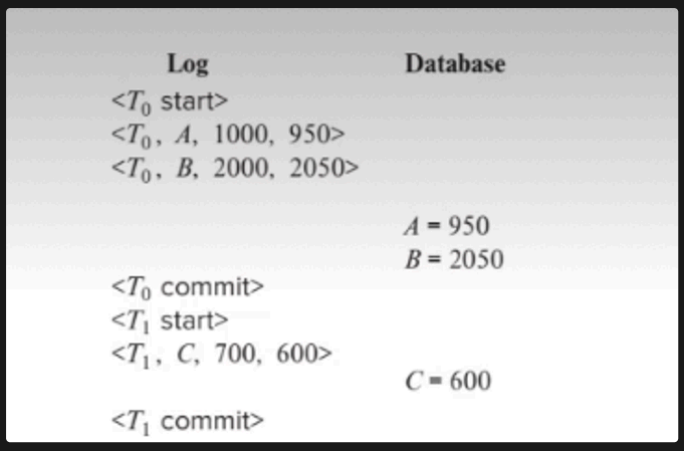
Log-Based Recovery

- A **log** is a sequence of **log records**. The records keep information about update activities on the database.
  - The **log** is kept on stable storage
- When transaction T_i starts, it registers itself by writing a
  $<T_i$ **start**$>$ log record
- *Before* T_i executes **write**(X), a log record
  $<T_i, X, V_1, V_2>$
  is written, where V_1 is the value of X before the write (the **old value**), and V_2 is the value to be written to X (the **new value**).
- When T_i finishes it last statement, the log record $<T_i$ **commit**$>$ is written.
- Two approaches using logs
  - Immediate database modification
  - Deferred database modification.

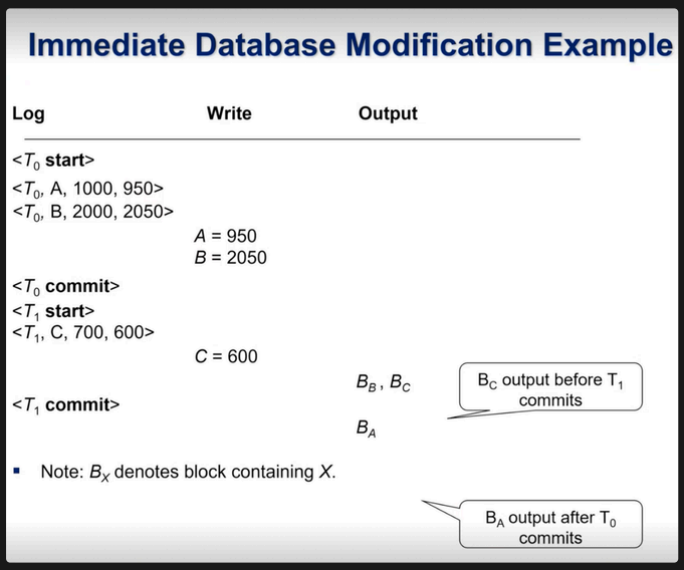
Transaction Commit

- A transaction is said to have committed when its commit log record is output to stable storage
  - All previous log records of the transaction must have been output already
- Writes performed by a transaction may still be in the buffer when the transaction commits, and may be output later

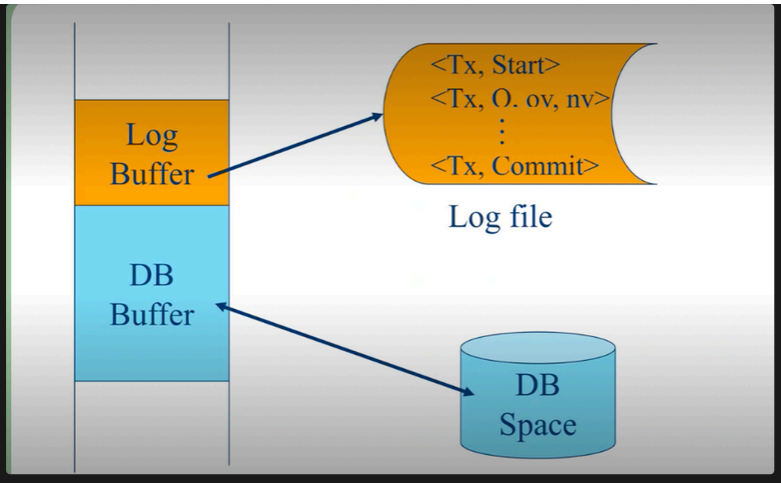
  - output can be done after commit
  - output can be perform after or before commit



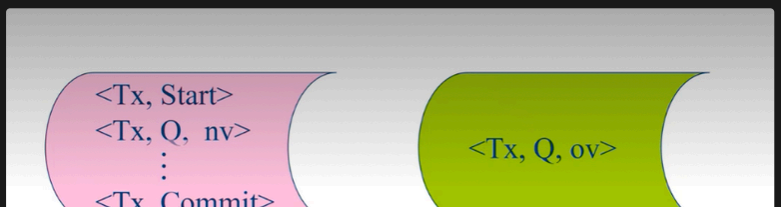
- This figure is a little bit misleading
  - it look like database is modify after the log
  - log → modify → commit (it look like we have to modify database first before commit) but in fact, log record must go to log file first, DB buffer go to db space
    - it show look like we have to output before commit → but actually, output can be done after commit.



Output may take place before or after commit.



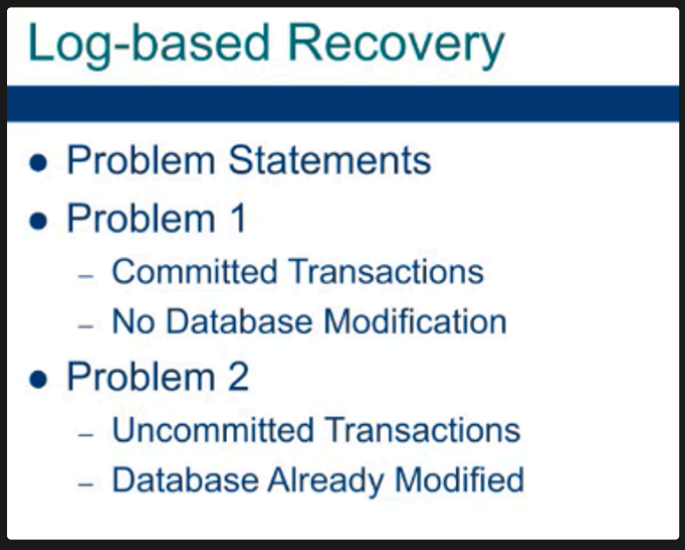
in here, <Tx. O, ov, nv> is in the same log file, but actually it possible and quit popular that we separate old and new value



Transaction Recovery (2)

- commit is an SQL command, and to make sure we can recovery all those committed transaction → we need to have an log record commit (which said that that transaction actually committed)
    - log record that reflect that the commit activity has been performed by user and application, so when you said commit, the system may not has yet return anything → it create commit record in a log buffer, and after a while that commit record output to log file (output successfully → return prompt)

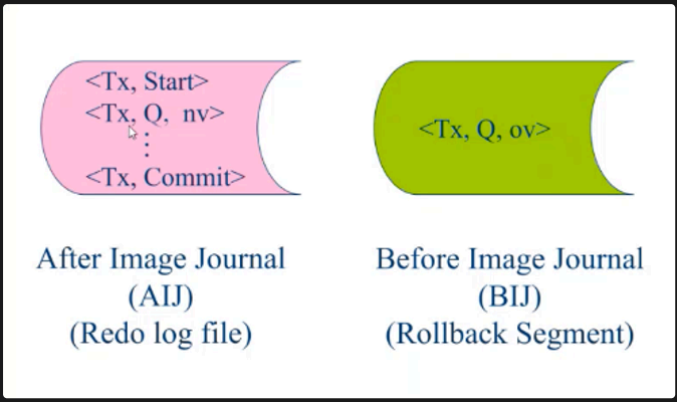
Recap From Previous Class



- Log based recovery
- 2 problem
    - committed transaction, not yet output
        - if data still busy in main memory, we normally not move anything to the disk
        - How to recover those committed transaction whose data are loss in main memory.
    - uncommitted transaction, output
        - maybe not enough room in memory → output before commit.
        - How to undo those uncommitted modification out of DB Space.

Log file, in principle, should be in stable storage

- We can also separate the old and new value



- Why do we sometimes separate them? (we can keep them together too)

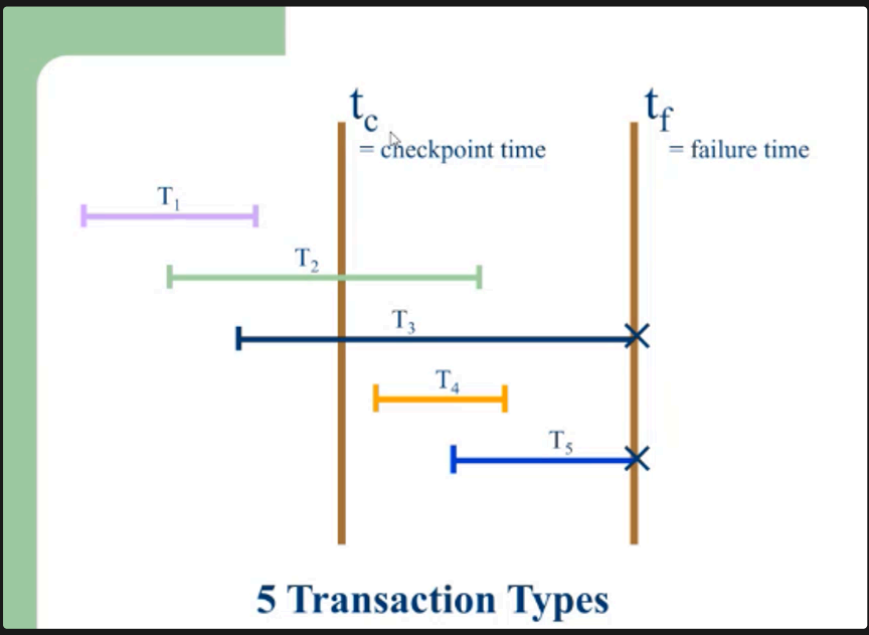
For those failed → system search back log file → can't find the commit record (transaction id placed in undo list) we use old value to undo DB Space, solve problem number 2

Those has commit record → transaction id place in redo list. we use new value after each operation in transaction to redo DB space, solve problem number 1

The point is we don't know which one already output → the point of check point

- check points are required to determine transaction that have already outputs changes to DB space.

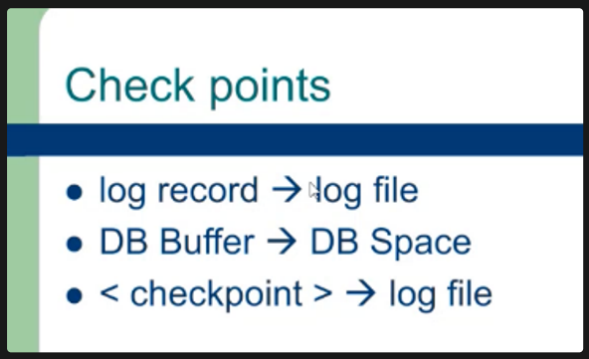
Content



- at checkpoint, we dump all modified buffer back to the DB space (we can't move directly, we have to output buffer to db space after the log)

Sequence of checkpoint

- if something went wrong, we could recover from log file information.
- in a log file, we have checkpoint log record (it also said checkpoint sequence number, and actual time of a check point)
  - in some system, check point record also show transaction id of those active transaction




at checkpoint time, T2 and T3 still active

- all those modified buffer of T1 is already output → no need to recover
- first part of T2 and T3 (until checkpoint) confirmed to already output to DB space.
- T3 and T5 failed.
- T4 → not confirmed to be output at all

- T3 and T5 failed → start recovery process.
  - use old value of T3 and T5, undo everything (from failure time up to the beginning)
- T2 → redo from the checkpoint time until its commit
- T4 → redo the whole thing


with the checkpoint, recovery time is reduced, we don't have to redo from the beginning, just redo from the checkpoint.

- checkpoint more frequently → shortened the recovery time
- How often checkpoint → you can set
  - if we not set, system has it own default
- Nowadays, system not failed quite often.

> at checkpoint, database buffer are forced output to DB space.

before checkpoint, the content in database buffer can output anytime, but we don't know

---

Some use case

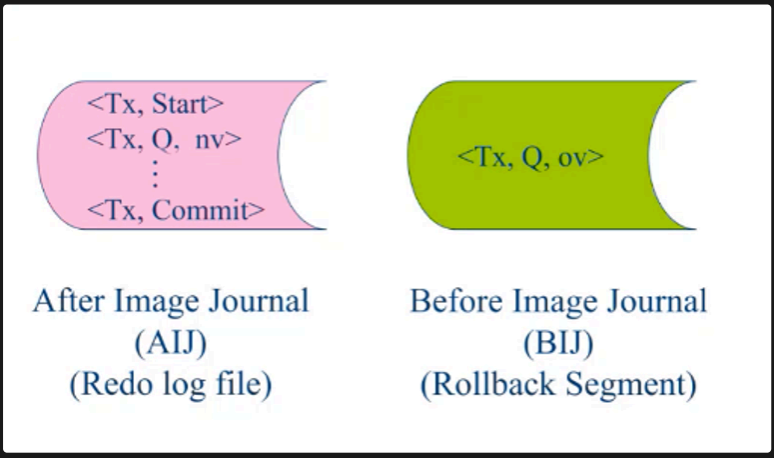
System not failed often. We purchased a new DB Server → staff complain, it failed often → 4 times a year, staff complain already. Another machine, after 2 years never failed. Back to early year, our first machine failed almost every year. (because power system around here not really stable.) (They gave priorities to factory around here) (Quite often there are thunder storm)

- Why we don't have UPS system → UPS cost more than database server.
- Because of failure, staff ask to buy UPS
- Dean ask why we need an UPS
  - UPS stands for **uninterruptible power supply**. It's a device that permits a computer keep running for a few hours during a blackout or when the primary power source is lost
  - What the main point of having UPS → staff answer him that we need UPS to avoid the damage to the hardware → unstable power could damage the hardware
    - Dean reply back you need a voltage regulator (stabilized power and voltage). → really good voltage regulator too

---

What is advantage of separating old and new value?

- it can be together, but we can also separate them.
- Most system follows this idea.



originally, it was about space. Based on separate log file like this we can recycle old value after each commit. BIJ size is quite constant size and quite big, big enough to handle transaction

- ov in BIJ for undo DB space → กรณี output before commit
  - we need old value because we want to recover uncommitted transaction that failed.
  - after transaction commit, old value no longer used
  - if we separate old value, new value like this → after commit old value can be recycle.
  - (if you combined them, even after commit you still have to keep old value)
- nv in AIJ for redo DB space → กรณี commit before output
  - we have to keep all those committed record even after commit, because we need new value to redo in case it failed after commit.

> **We have option to install one of them or both, and still for each option you can still have committed and rolled back logically as usual.**

What if the BIJ is full?

- for a transaction that involved lot of data, you may update lot of data, and you need to keep old value in a log file before commit. What if too many that BIJ can't handle.
  - blood group case (20 M → but your BIJ can handle only 1 million)

The transaction that causes the BIJ full problem will be rolled back.

👉 once, there are computer center that try to run that big transaction over night → 40 instructions → the log file is full → system rolled back → example of transaction failures

(Already mentioned)Solution → those 40 instruction are not related, they should not be in the same transaction → set autocommit on.

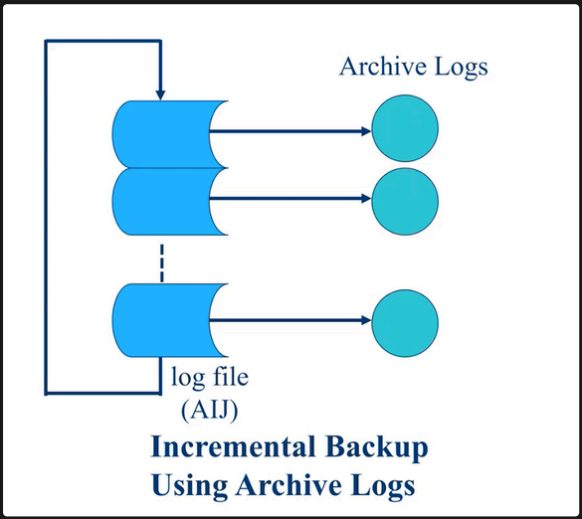
(Solution)

- (not reccomened) bypass the BIJ (no log option)(not reccomened)
- enlarge the BIJ (online?) (some system allow you to do it online, you can add space to running BIJ real time without shutting down)
- (recommened) separate a big transaction into smaller ones
  - if the instruction is not related. → Fine
  - But if they should be together.
    - ex: blood group update
      - maybe update 1 province at a time, eventually the entire country blood group will be finish.

What if the AIJ is full?

(use multiple AIJ)

- AIJ keeps commit record, so if it full and the system continue working as usual
  - failure occur → you can't recover (no nv for recovery)→ you loss the entire system.
  - some system stop working to prevent that
- Some DBMS stop working (MySQL, Microsoft, etc.)
- Some DBMS continue working (Oracle in the past)
- Expensive system let the machine run, small work group stop working
- **How to solve this problem (Solution)**



- instead of having only 1 log file, they have multiple AIJ nowadays, so they have 1 active AIJ at a time.
- if the first one is full, the system switch to the next one, and so on.
- You have to estimate how many log file(AIJ) you need in a day.
  - In an unmanned system, no attendance, they set up 8 tape drive, and someone change the tape every morning.
- For back up → we still use tape media **(reusable after certain time)(archived media, archived backup)**
- the idea is relaible source to keep the data → tape archive (ok)

👉 The solution for when AIJ full is to use multiple AIJ, first one full → next one. And if we don't do anything(don't archive), it will rotate back to the first one and replace the first one.

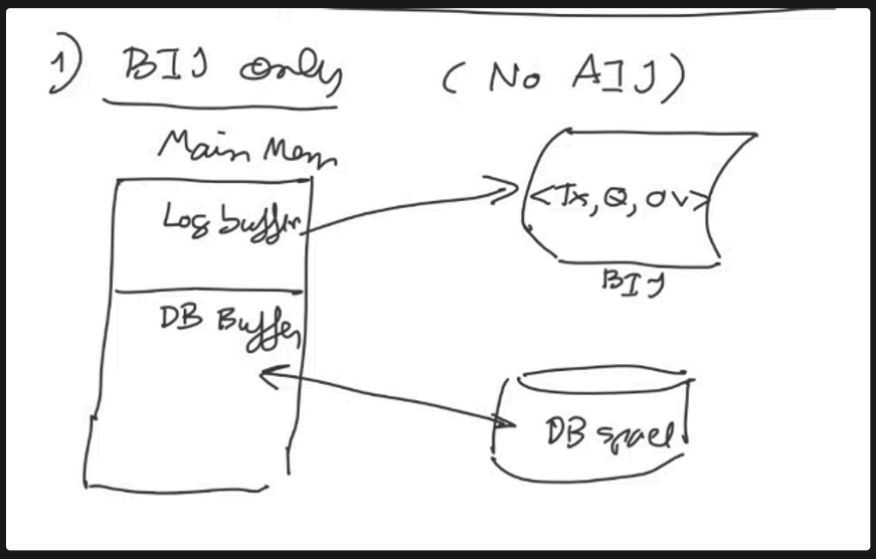
- one log file rotate back to itself → nooo, at least have 2.
- if you have people attend system all the time → you can have only 1 such archived device, and when the first AIJ is full → you archived it to your archived media. And at that time, the DBMS will use another
- in bank, they use robot to change a tape.

This not only solve the AIJ full problem, it also called incremental backup using archived log. (archived log for AIJ only or for combined)

- Why do we need to archived them?
  - in case of transaction failure or system crash → you can use log record
  - to recover the system incase of disk failure or DB space corruption.
- we have transaction failure, system crash, and disk crash
  - for transaction failure and system crash → we don't need (m?)any backup
  - for media crash → we need backup (the archived log)
    - we need it for disk crash, without the archived log, we at risk.

Log based Recovery Classifications (based on availability)

1. **BIJ only (No AIJ)**
    - one of the most popular → the vendor don't want customer to call them when AIJ is full
    - you don't have new value and commit record
    - you record only old value without new value



    - commit, rollback works as usual logically
    - The point is → is How (you don't have AIJ, how does it works)
        - how commit work
            - we don't even have AIJ to put commit record
            - without AIJ, transaction commit mean the modified buffer of that transaction must already be output to the DB space completely.
            - when you sit in front of your. machine using SQL, update insert delete and then you said commit, the machine return prompt? what happen physically without AIJ? Your modified buffer must be completely output to DB space.
        - how roll back work
            - old value from the BIJ can be used

Side effect (AIJ is missing)

💚 Advantage
- No AIJ full problems
- Save some space

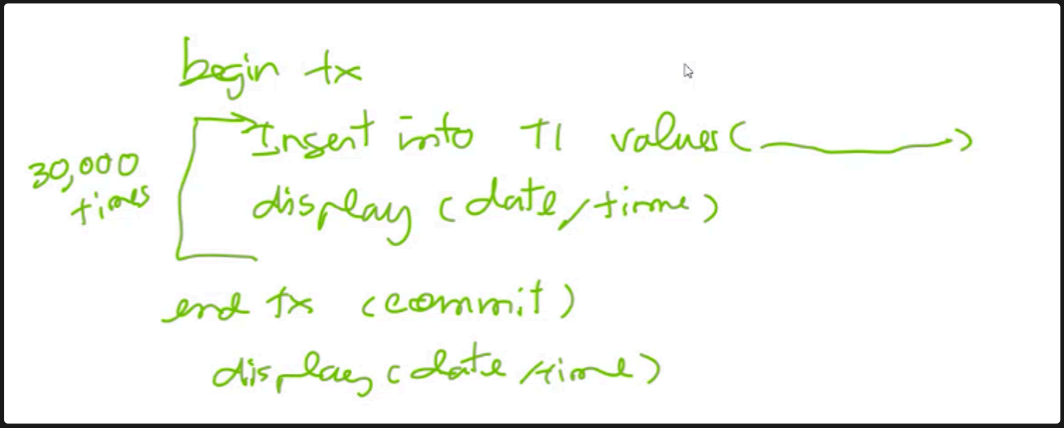
- BIJ can also be full, when BIJ full → that particular transaction roll back, not the entire system
- some company throw away machine when log file is full → they don't know about transaction thing

❤️ Disadvantage (You don't have AIJ, something must be wrong)

- it about commit (commit now mean → the modified buffer of that transaction must be completely output to the DB space, aj. don't want to use the word output because some system, they don't force output. They just let the output go naturally. → And it could take many hours → Commit can be very slow — It takes time for the data in the database buffer to be completely output to the DB space
  - (The DBMS and OS determine when to output (not the application))
    - when you said commit → not commit immediately.
- We don't have AIJ → no checkpoint

- consequence of slow commit, during transaction before commit(end of transaction), your transaction may use many data and the system may lock some table/data. So if your transaction have slow commit, people wait more. Wait for you to commit because before commit, you lock resources. If you commit quickly people can use resource more quickly and throughput is better.

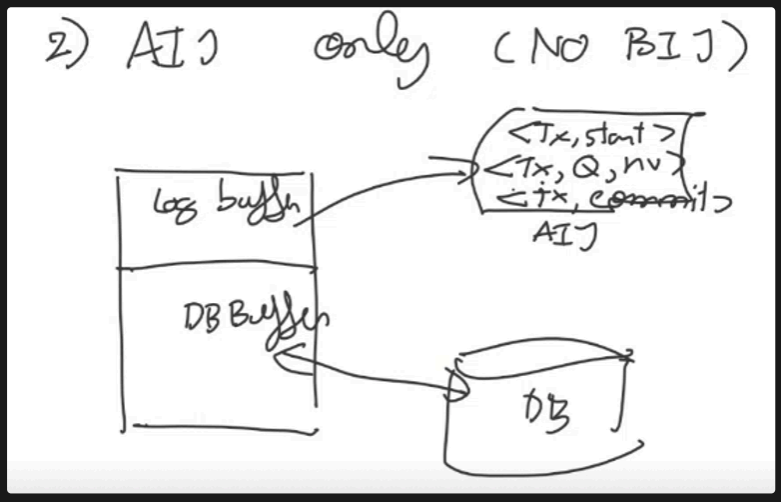
👨 Use case: a vendor install a system free of charge to a potential customer, the customer wrote the small program



- the commit time between last insert and the commit took them 18-19 hours (test system).
- insert 30,000 rows took 18 hours to commit.
- they try it on biggest machine from that vendor took 5 hours.
- Problem → if you have to long commit time → your system doesn't have an AIJ.
  - with an AIJ, the commit record go to AIJ → finish
  - without AIJ, depend on DBMS and OS.

- Solve → Try to create AIJ and check if it work → when added 19 hours went down to 8 minutes.
    - at first they refused → they said with log file, you have to output to both log and disk → double or more I/O. so they said it will be slower → not true.
    - in reality, to output log record it just like add new block to the end, but to move data from DB buffer to DB space require careful placement. you have to put in correct way so you can retrieve them easily later, require correct placement and index. So it take more time that output log buffer to log file.
- with log file AIJ, commit a lot faster.

2. **AIJ only (No BIJ)**

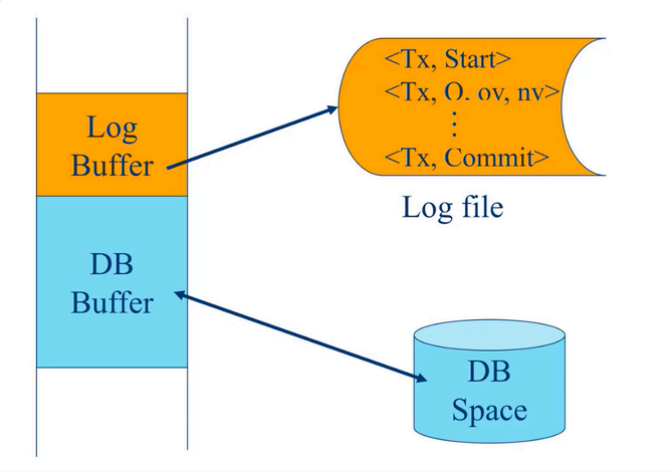


- commit and rollback work as usual logically
- How it work?
    - commit → (as usual) transaction commit go to log file
    - roll back(no old value how does rollback work) you don't have any old value to undo. How to solve second problem (output before commit)
        - we don't know when it was last update, maybe 10 years ago?
        - (Prevention) do not allow output before commit
            - this kind of DBMS won't allow output before commit
            - Old value is not required.

Recovery and Buffer Management (Log based recovery)

Quick Review

- Last time before a midterm, we have log based recovery topic.



  - idea is quite simple, we keep log record.
    - When we start transaction, we have <Tx, Start>
    - When the transaction is active during processing of transaction, we have <Tx, Q, ov, nv> (old value, new value)
    - When the transaction is committed, we have <Tx, Commit> output to the log file.
      - So the physical commit(when we have log-based recovery) → when commit record output to the log file successfully.
  - Based on this principle, During recovery time after failure, the system simply search commit record from the back
    - if the commit record is found → that transaction commit
    - otherwise → the transaction failed
  - Even though, you have commit record is log buffer → it not yet commit (At failure time, the log record in log buffer maybe lose) → we count only commit in log file or commit in AIJ

> 👉 Old value/ new value
>
> - we use new value to redo → solve first problem (commit before output)
> - we used old value to undo DB space → solve second problem (output before commit)

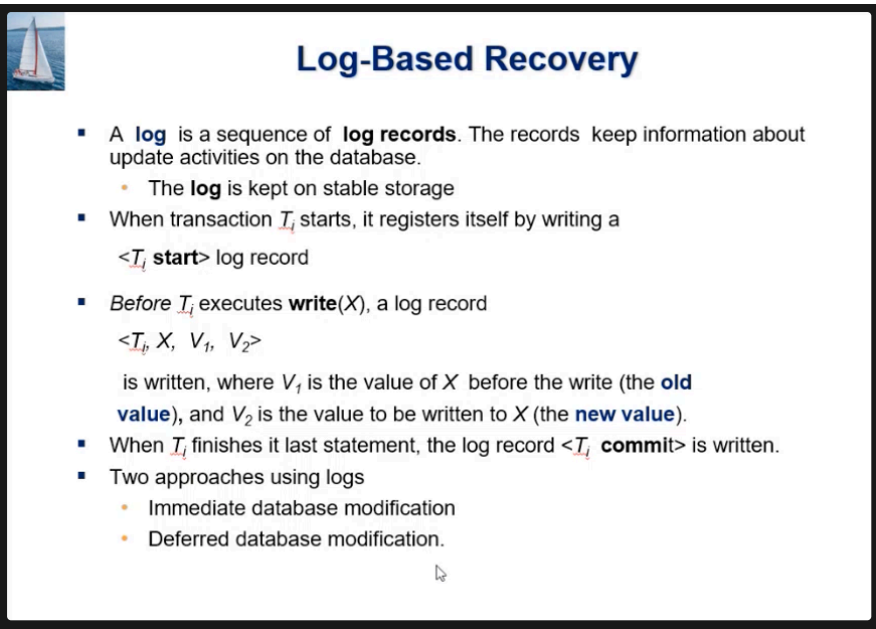
Old value is store in BIJ, new value stored in AIJ → can choose to install one of them or both of them

---

> **Content**

🌱 2 Approaches of log-based recovery

(Book has slight different terminology, same idea but different terminology)

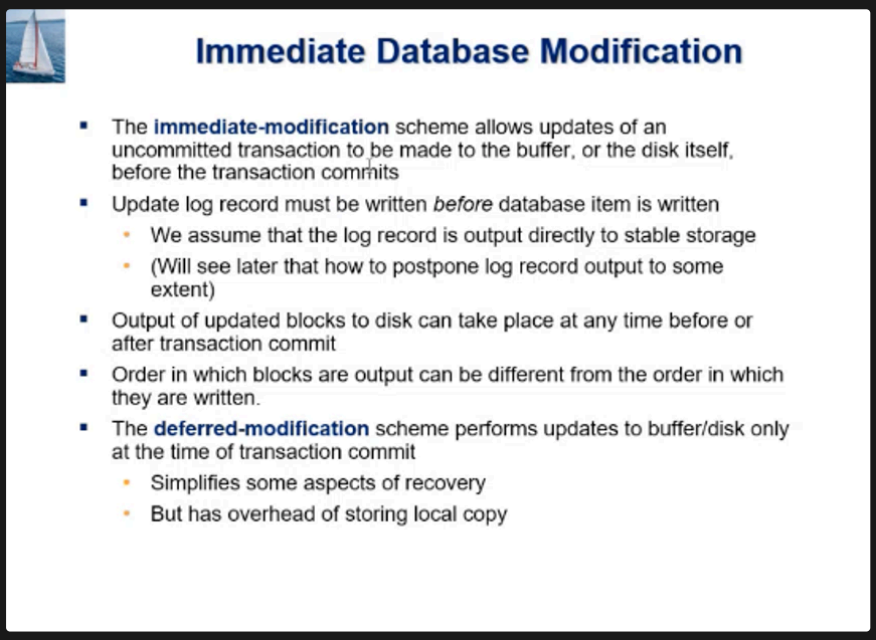


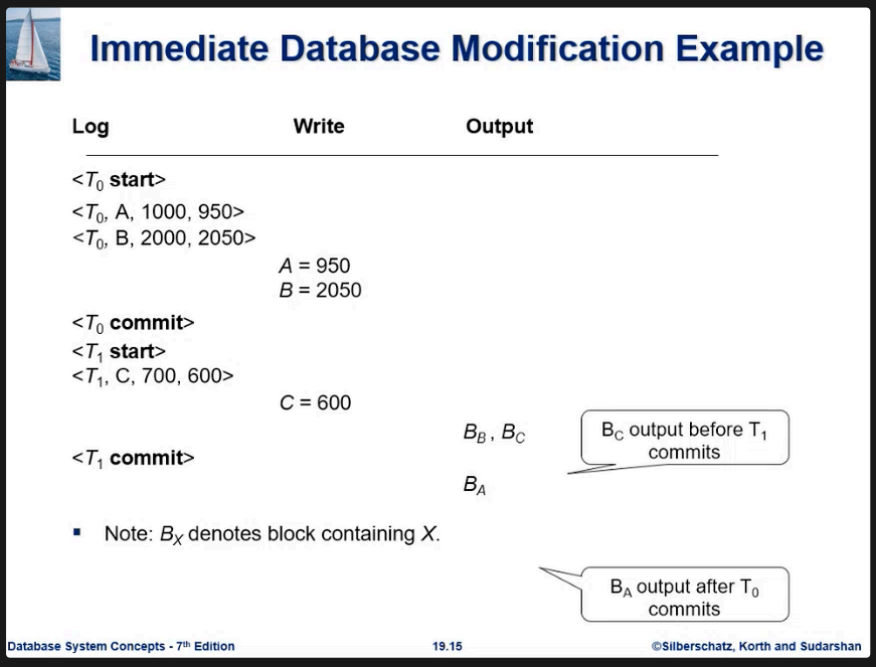
From book

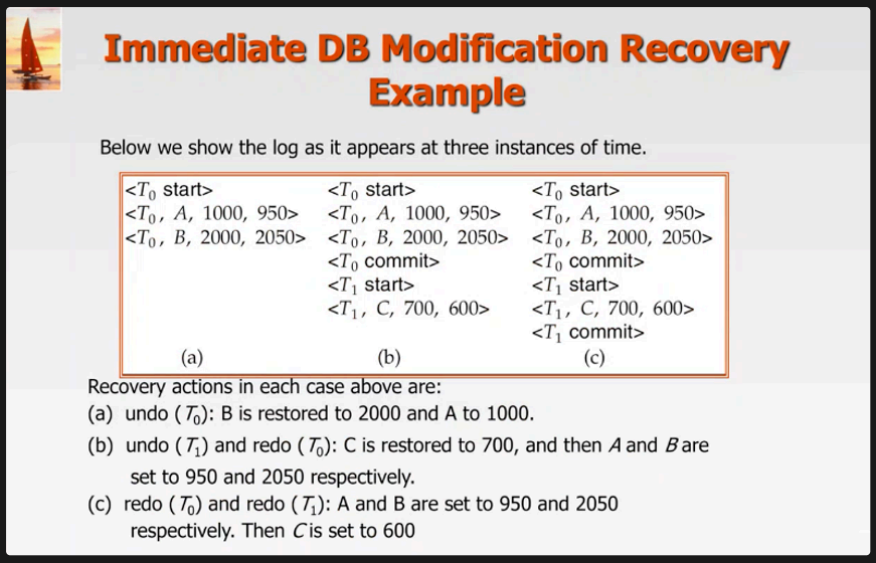
- There 2 approaches using log (log based recovery)

1. **Immediate database modification**
   - The immediate-modification scheme allows update of an uncommitted transaction to be made to the buffer, or the disk itself, before the transaction commits
   - Output may take place before or after commit

👉 - in this categories
- BIJ only is an immediate DB modification (take place before commit)
  - no new value → must output before commit
- Both AIJ & BIJ is an immediate DB modification (output anytime)



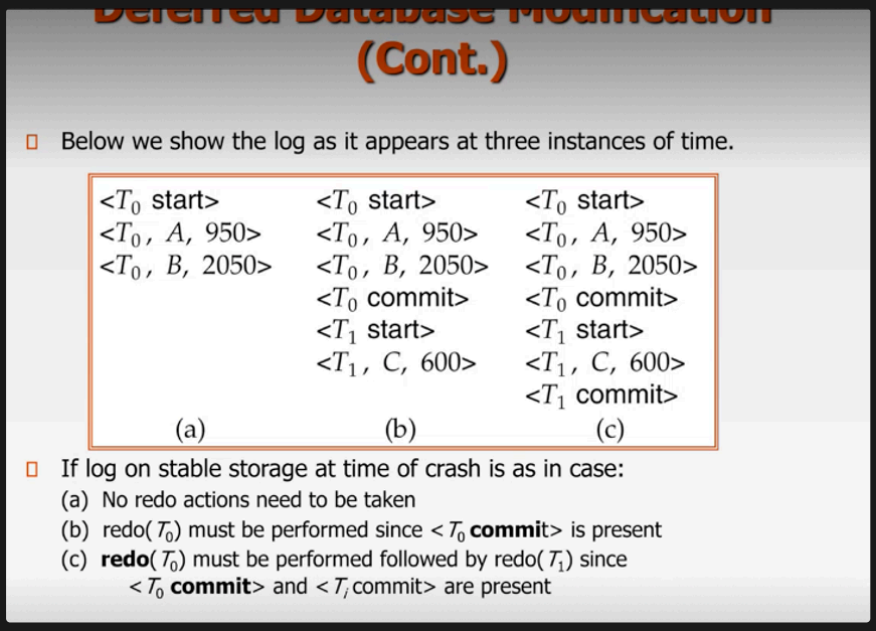




- in immediate database modification, we need an old value, because immediate imply that output can be done before commit, so we need old value for recovery process (undo)
- this diagram show both old and new value

2. **Deferred database modification (can be postpone)**
   - The deferred-modification scheme performs updates to buffer/disk only at the time of transaction commit
   - It must commit first - commit then later output
   - Output may take place after commit or at the commit time

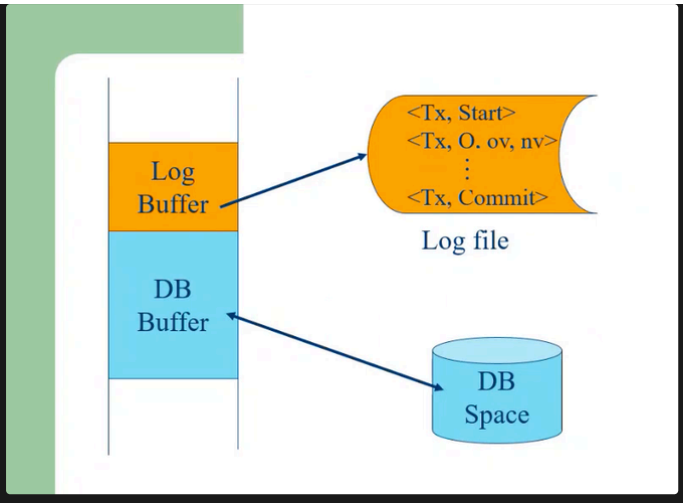
> 👉 AIJ only is deferred DB modification (we dont have old value, not allow db buffer to output to db space before commit)



   - in this, it has only AIJ (new value)
   - Transaction T0 transfer 50 dollars from Transaction A to B
     - Before the transaction A was 1000, B was 2000
     - After transaction A become 950, and B become 2050
     - The log only show new value (AIJ)
   - For transaction T1, we withdraw 100 dollars from C (C was 700)
     - After withdrawal → become 600

Show only new value

🌱🌱 Buffer Management (19.5 - book)



from, this we see log buffer, db buffer and know that they are in main memory

- Log buffer and DB buffer are in main memory
- **Who manage the buffer (DB buffer and log buffer)?** Should it be manage by the DBMS or the OS?
  - when we study OS → we know that OS controls the main memory. It can paging, segmentation, ..., It also has page replacement strategy like FIFO, read recently used, read frequently used, ... that adopt by the OS
  - OS actually control the main memory, but should we let OS handle buffer management of this log buffer and database buffer?
  - (There were discussion and also implementation.) How much involvement should DBMS be in this buffer management activity?
    - after long discussion and several implementation, it is very clear that it is the DBMS that should be in charge of this buffer management.

> 👉 There are protocol that the DBMS must execute and must guide, force DBMS to follow **such protocol.**

Those Protocol?

(Some topic from book)

1. **Log-Record Buffering** - Log record are buffered in main memory, instead of being output directly to stable storage and it need to be managed.

Log Record Buffering

- **Log record buffering**: log records are buffered in main memory, instead of being output directly to stable storage.
  - Log records are output to stable storage when a block of log records in the buffer is full, or a **log force** operation is executed.
- Log force is performed to commit a transaction by forcing all its log records (including the commit record) to stable storage.
- Several log records can thus be output using a single output operation, reducing the I/O cost.

- Log force is performed to commit a transaction by forcing all its log records (including the commit record) to stable storage.
  - This book always assume that log file is in stable storage → so it mean force to log file. (when we say force to stable storage)
- Several log records can thus be output using a single output operation, reducing the I/O cost.

👉 There an important protocol on **Log Record Buffering**

Log Record Buffering (Cont.)

- The rules below must be followed if log records are buffered:
  - Log records are output to stable storage in the order in which they are created.
  - Transaction T_i enters the commit state only when the log record <T_i **commit**> has been output to stable storage.
  - Before a block of data in main memory is output to the database, all log records pertaining to data in that block must have been output to stable storage.
    - This rule is called the **write-ahead logging** or **WAL** rule
    - Strictly speaking, WAL only requires undo information to be output

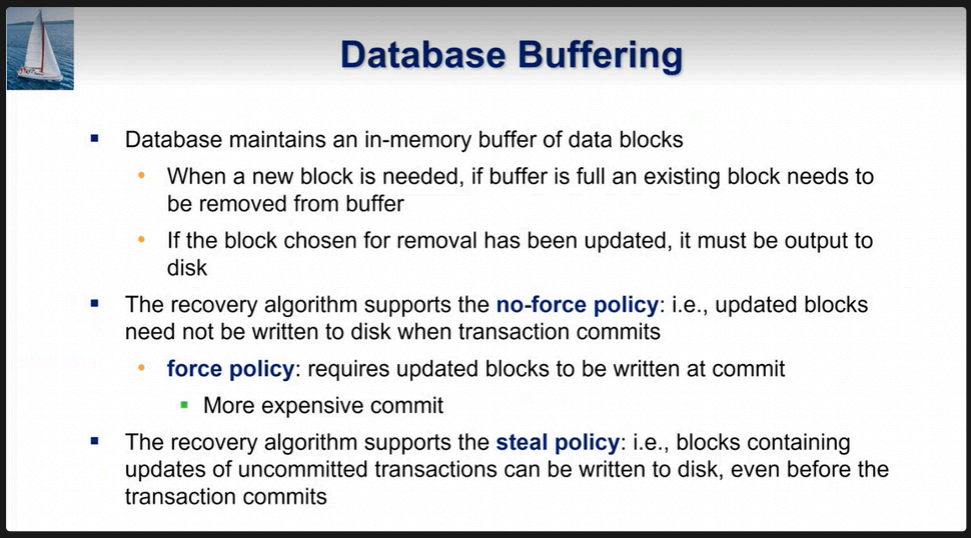
<u>Rule of Log-record buffering</u>

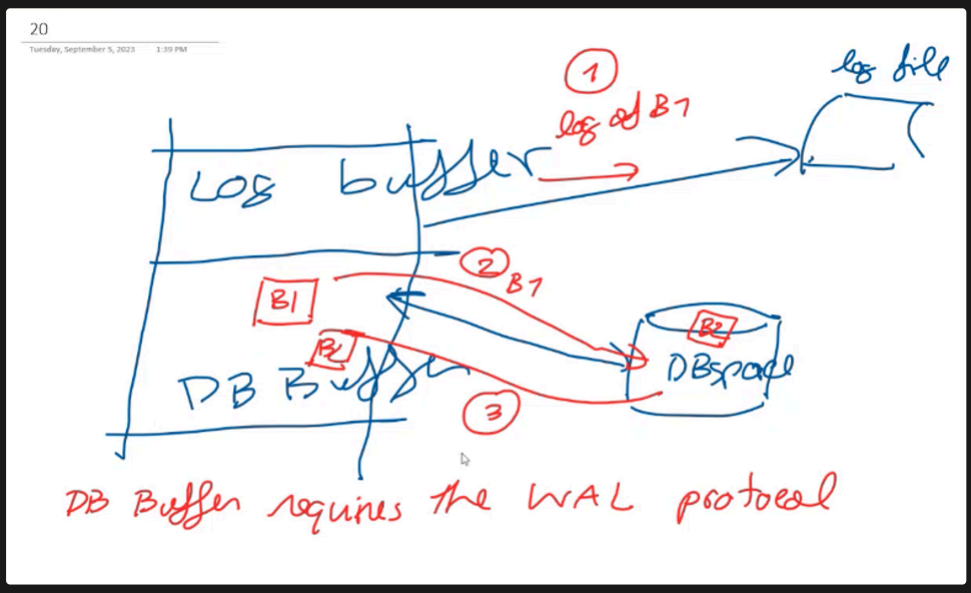
- Log records are output to stable storage in the order in which they are created
  - this imply that commit record must be the last one of that transaction.
- Transaction Ti enters the commit state only when the log record <Ti commit> as been output to stable storage. (to ensure physical commit)

- WAL → before block of data output to the database, log record output to stable storage first.

> 👉 The DBMS must follow this rule
>
> - For OS point of view, DBMS is just an application program for OS
>   - So, this is application requirement of DBMS for log based recovery.
> - It must be DBMS that instruct the OS that ok, you must go now or wait → they must be test together.

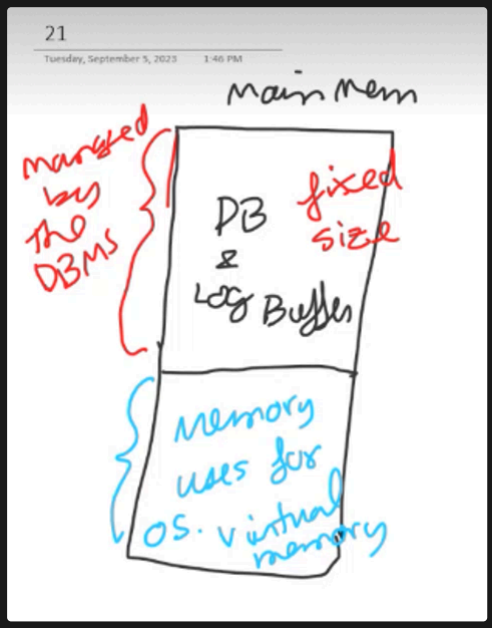
2. Database Buffering





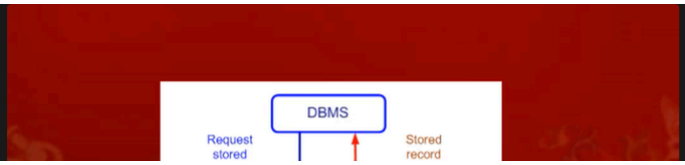
- Suppose you have block b1 in DB buffer and a block b2 in DB space → in practice, after a while your database buffer gonna be full because you use data (input data to buffer), and if you want b2 to be input in DB buffer, you can't just input b2 directly to buffer → b2 has to replace a block here (DB space is full). → bring b1 out before you move b2 in
- Step (database buffer simple protocol)
  - When you bring b1 out, you need to output the log of b1 first to the log file
  - b1 back to DB space
  - b2 in to db buffer
- DB buffer also requires the WAL protocol

3. **Operating System role in buffer management**
    - We can manage the database buffer by using one of two approaches:
    1. The database system reserves part of main memory to serve as a buffer that it(DBMS), rather than the operating system, manages.
        - DBMS reserves part of main memory to serve as a buffer that the DBMS itself manages



        - From main memory pov, the os allow the DBMS to reserves part of the main memory as database and log buffer
        - size of database and log buffer is fixed size → The DBMS occupied a fixed size of main memory and it's managed by the DBMS.
        - It's the DBMS that manage the protocol without passing through the OS.
        - blue part → virtual memory
    2. The database system implements its buffer within the virtual memory provided by the operating system.
        - it is the DBMS that implement it buffer within the virtual memory provided by the OS.

Diagram show interaction between DBMS and OS.

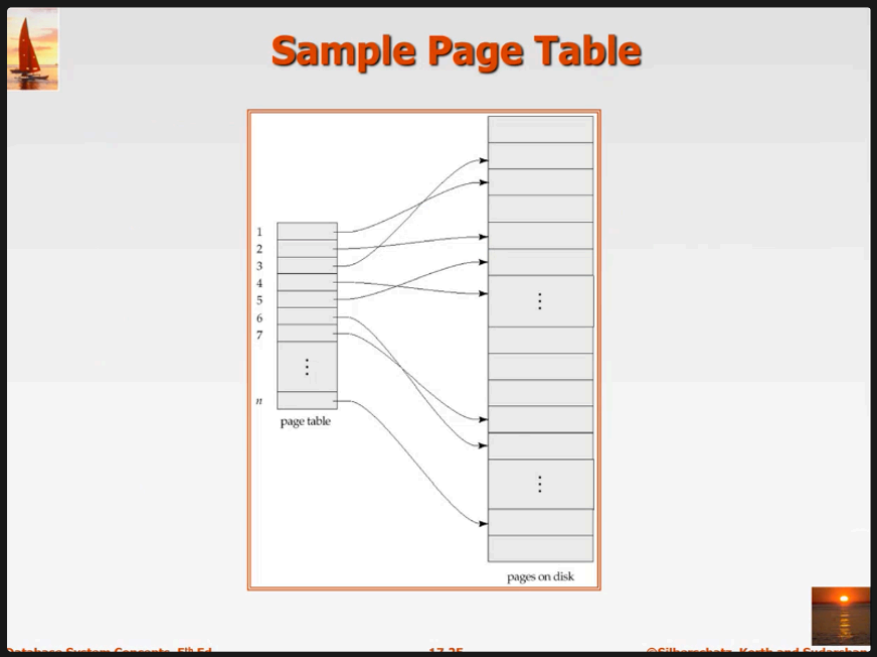

DBMS
Request stored
Stored record

Shadow Paging

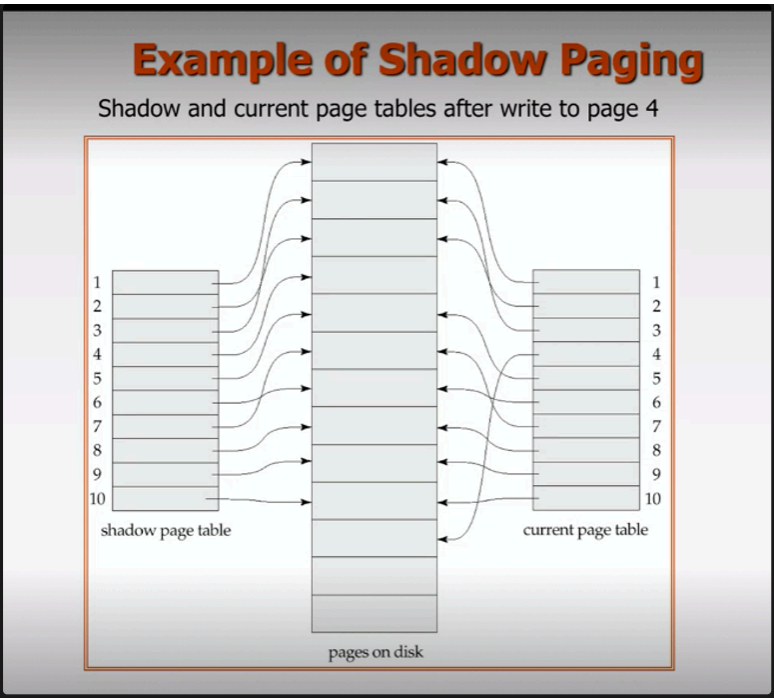
👉 log based recovery technique require recovery time (time to recovered), after system went down, it would take few minute or up to hour(for big system), what if we can afford to wait?

✅ Shadow Paging

- an alternative to log-based recovery
- allow immediate recovery
- this technique → small device (pc, handheld device), and also military system?
- minimum recovery time → no need to undo, redo



- our database will be separate into pages, and you need directory call page table
- page table have a pointer, point to pages on the disk
- this pages handle by DBMS
- when a transaction start, and the database operation work on the data → a new page table will be created



- you have shadow page table, at first it only have 1 page table.
- once, you start transaction, the all page table become shadow page table.
- and the DBMS create new page table called current page table.
- look at this diagram, page number 4, the old directory point to a place (ด้านบน), but when we modified or update page number 4, we don't modified at old place, we copy to new area, and we have pointer point to the new area.
- idea → pointer point to page in database and when any update take place, you don't modified existing one, you copy the content to a new area, and modified a new area.
- Modification take place in a new page
- System failed before commit → you restart the system, it will use shadow page, and point to the old place (switch to old data) → immediate roll back
- commit → save the current page table and discard shadow page.

👉 this one work for small system(with minimum update), not very good for big system with alot of update.

Transaction Scheduling

> Content

🌱 Transaction Isolation

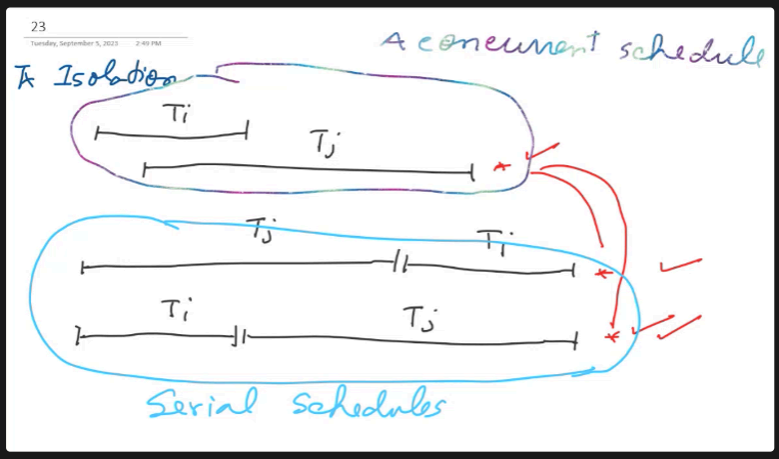
(about multitasking, concurrent execution)

(we can handle multiple transaction during same period of time, because the speed of i/o is much slower than speed of cpu, so we can handle many transaction that is basically i/o bound → use more i/o time than cpu)

- But the challenges is how to obtain correct result?

Let moved back to ACID properties: Isolation

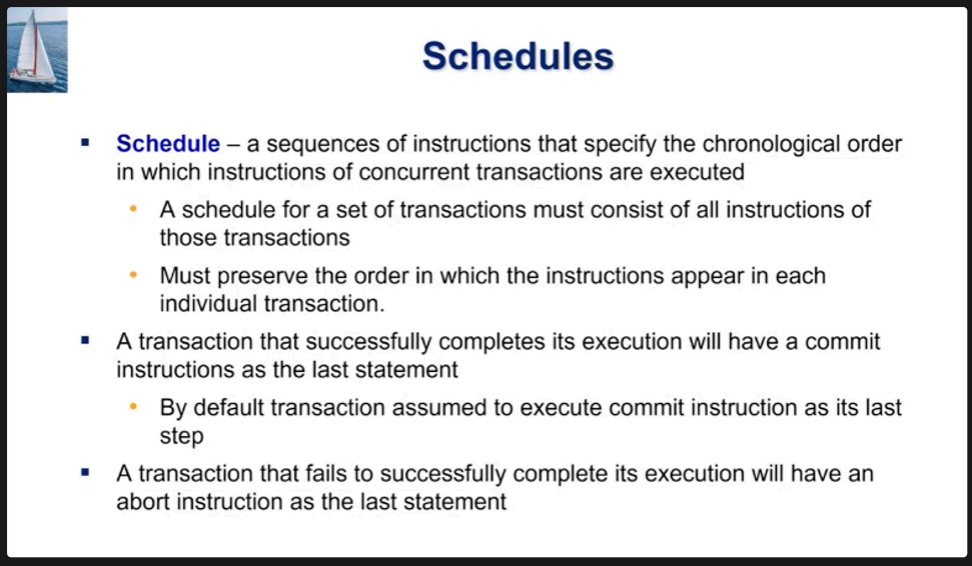
- Isolation: Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions.
  - Intermediate transaction results must be hidden from other concurrently executed transactions



  - The last approach is most popular, but in practical the second is also accetable because it just a fraction of second, we don't really know which is which.
  - The second and last one called → serial schedule

👉 We want concurrent schedule that give the same result as serial schedule → we call that a concurrent serializable schedule

What is Schedule?



chronological order → order according to time.

A schedule is an arrangement of instructions execution. There are several transaction in a schedule. Each transaction comprises several instructions.

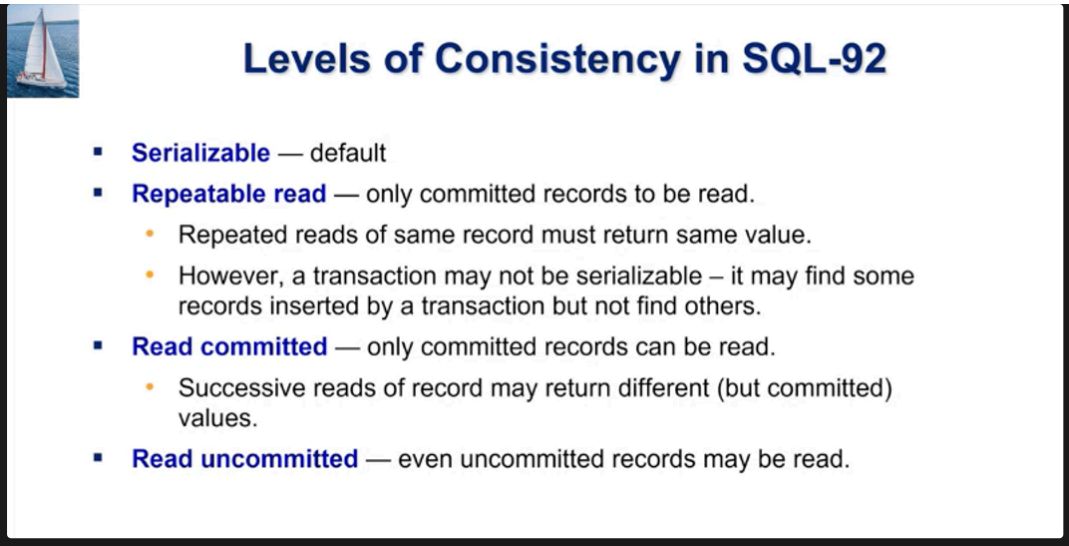
- Schedule is basically a sequence of instruction execution (execution arrangement)
- 1 schedule typically comprised of many several transaction → each transaction comprised many instruction.

- **Serial schedule** → instruction that belong in the same transaction is execute uninterrupt from the beginning to the end of transaction.
- **Concurrent schedule** → we may execute instruction from different transaction, sometime we executed from the first tx, and switch,....

👉 In practice, we don't need serial schedule, we need concurrent schedule, however we need concurrent serializable schedule.

> Serializable schedule(concurrent that gave the same result as serial schedule) ≠ serial schedule

🌱🌱 Level of Consistency



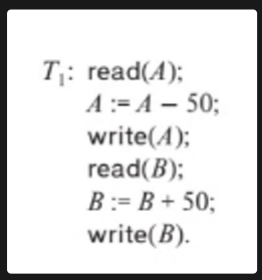
- we don't have to be serializable at all time → can change
- you can adjust to be anything
- In practice, many people not set their system to be serializable, and when ask why, they reply something like we don't want our transaction to be run in sequence. They misunderstood the term serializable and serial

> 👉 if we choose serializable → it mean conflict serializable schedule(17.6 not 17.8) (most of the time)

✅ 2 samples transaction

According to the book, the book gave us 2 sample transaction

1. Transaction comprised only sql statement (2 update statement)



- T1: Money transfer transaction, transfer 50 dollars from A to B
  - read(), write() → update statement

2. Transaction comprised of both sql statement and application program statement (refer to program variable inside the source program

T_2: read(A);
$temp := A * 0.1$;
$A := A - temp$;
write(A);
read(B);
$B := B + temp$;
write(B).

- T2: Read A and reduce 10% from A using temp, write A, read B, add that temp to B and write B
  - how to implement → calculate temp first
    - read that row A → calculate the temp in the program, you read row B reduce temp from A, add temp to B and write back(can be done using embeded sql)
      - so in embeded sql → you select particular row, modify the value and write back to the row.

✅ Example

> we want concurrent schedule that give the same result as the serial one

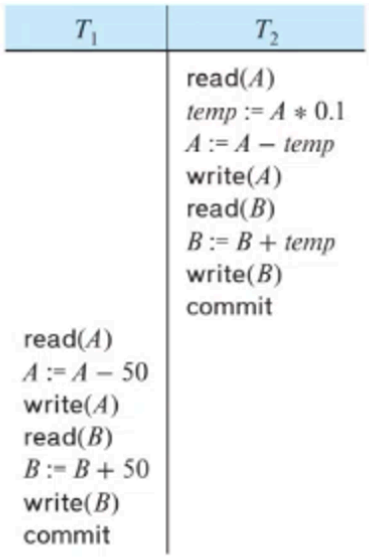
🌱 Diagram 17.2

T_1	T_2
read(A)	
$A := A - 50$	
write(A)	
read(B)	
$B := B + 50$	
write(B)	
commit	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)
	commit

Figure 17.2 Schedule 1—a serial schedule in which T_1 is followed by T_2.

- A serial schedule
- T1 run until commit, and then follow by T2

🌱 Diagram 17.3



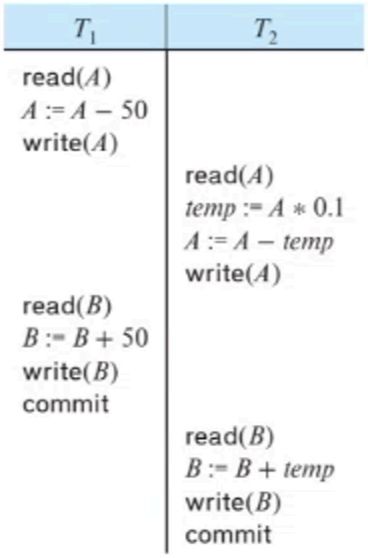
T_1	T_2
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)
	commit
read(A)	
$A := A - 50$	
write(A)	
read(B)	
$B := B + 50$	
write(B)	
commit	

Figure 17.3 Schedule 2—a serial schedule in which T_2 is followed by T_1.

- A serial schedule
- T2 run first, follow by T1

🌱 Diagram 17.4

- Concurrent schedule that gave the same result as 17.2
  - you can move read(A), write(A) of T2 down and move read(B) and write(B) to T1 up



T_1	T_2
read(A)	
$A := A - 50$	
write(A)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
read(B)	
$B := B + 50$	
write(B)	
commit	
	read(B)
	$B := B + temp$
	write(B)
	commit

Figure 17.4 Schedule 3—a concurrent schedule equivalent to schedule 1.

- concurrent schedule (not a serial one)
- this one is the concurrent schedule that give the same result at 17.2
  - but how do you know, like us we human can be able to tell, but how dbms know (this is our challenge)

- we know that this is same result and correct as 17.2
  - we considered the calculation to decide, but the problem is for t1 is okay, but for t2, t2 has calculation in source program, it beyond the scheduler, dbms can't access source code. (or else one day, all run in the cloud)

🌱 Diagram 17.5(another concurrent schedult, give wrong result)

- we refer 17.4 because it gave the same result as 17.2

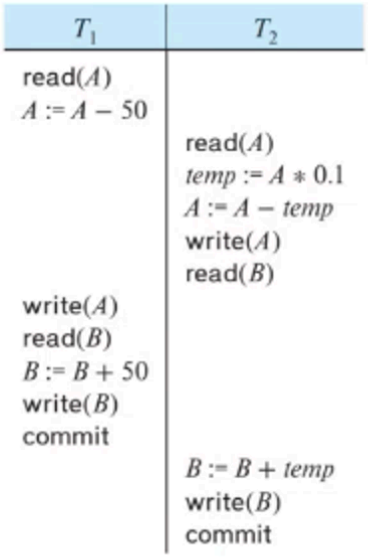


Figure 17.5 Schedule 4—a concurrent schedule resulting in an inconsistent state.

- a concurrent schedule (not a serial one)
- T2 write first before T1 → when T1 write (the value is incorrect) → missequence
  - we should be able to detect this without calculation.
- T2 should have read A which was produce by T1
- not conflict serailizable → can't transform to serial → because write (A) of T2 and write(A) of T1 is actually conflict.

How do we know that it correct (we can't determine by comparing result, some can't access all the code)

👉 **We have to have algorithm, we have to have a standard on what is correct and what is not.**

- the dbms know the read and write sequence, the dbms receive read andwrite sequence from application
  - from the read , write sequence → we'll be able to tell whether it correct or not(same result as serial schedule)
  - try to swap non conflict instruction

🌱 Diagram 17.6 (show only read and write)

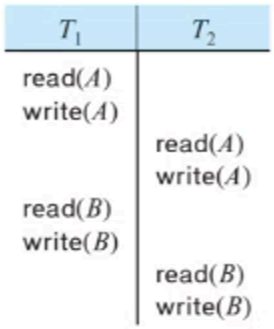


Figure 17.6 Schedule 3—showing only the read and write instructions.

- 17.4 with only read and write

🌱 Diagram 17.7

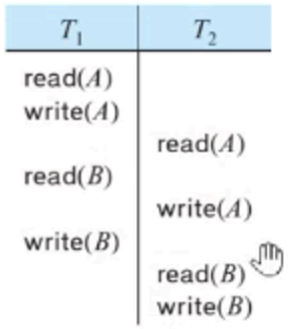


Figure 17.7 Schedule 5—schedule 3 after swapping of a pair of instructions.

- swap some instruction from 17.6
- still same result becase they work on different data item
  - write(A) first, read(B) first doesn't matter

🌱 Diagram 17.8

- swap instruction of 17.6 to have a serial schedule

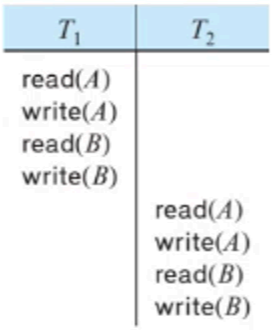


Figure 17.8 Schedule 6—a serial schedule that is equivalent to schedule 3.

👉 **Some Technical term**

- Conflict equivalent
  - if a schedule S can be transformed into a schedule S′ by a series of swaps of non-conflicting instructions, we say that S and S′ are **conflict equivalent.**

🌱 Diagram 17.9 (not a conflict serailizable schedule, gave wrong result)

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule < T_3, T_4 >, or the serial schedule < T_4, T_3 >.

- this gave incorrect result

🌱 Diagram 17.x (non conflict serializable that give correct result)

- A schedule S is **view serializable** if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but *not* conflict serializable.

T_{27}	T_{28}	T_{29}
read (Q)		
	write (Q)	
write (Q)		
		write (Q)

- What serial schedule is above equivalent to?
- Every view serializable schedule that is not conflict serializable has **blind writes.**

- this give correct result → same as serial (T27, 28, 29)
  - not conflict serializable→ can't swap write of T27 and T28
- if you run in sequence, the result of Q will be the one that write by T29

🌱 Diagram 17.13 (non conflict that give correct result)

- The schedule below produces same outcome as the serial schedule $< T_1, T_5 >$, yet is not conflict equivalent or view equivalent to it.

T_1	T_5
read (A)	
$A := A - 50$	
write (A)	
	read (B)
	$B := B - 10$
	write (B)
read (B)	
$B := B + 50$	
write (B)	
	read (A)
	$A := A + 10$
	write (A)

- Determining such equivalence requires analysis of operations other than read and write.

- produce same output, but not conflict equilvalent to serial

🥰 Standard of correctness (concurrency control based on correctness concept)

✅ Conflict Serializability

- so if a schedule S can be transform into schedule S′ by a series of swaps of non-conflicting instructions, we say that S and S′ are conflict equivalent
  - and if schedule conflixt equivalent to a serial schedule → we say that a schedule S is conflict serializable.
  - conflict serializable is standard of correctness → if we said this schedule is conflict serializable → it's mean it correct

Conflict Serializability (Cont.)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 3

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

Schedule 6

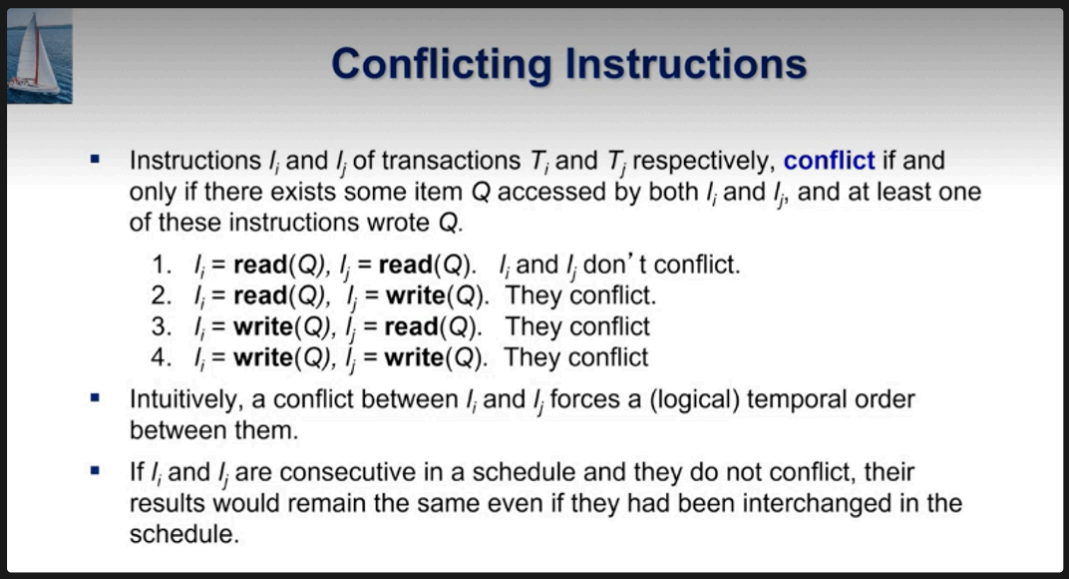
👉 we can said that 17.6, 17.7, 17.8 are conflict equivalent schedules.

and because 17.8 is a serial schedule, 17.6 and 17.7 are conflict serailizable schedule. (They are conflict equivalent to a serial schedule)

What are those non-conflicting instruction?

- we can swap read(A), write(A) of T2 with read(B) write(B) of T1, simply because the two work on different data item
- instruction that work on different data item not conflict, naturally

in the case, that the instruction from 2 transactions works on the same data item.

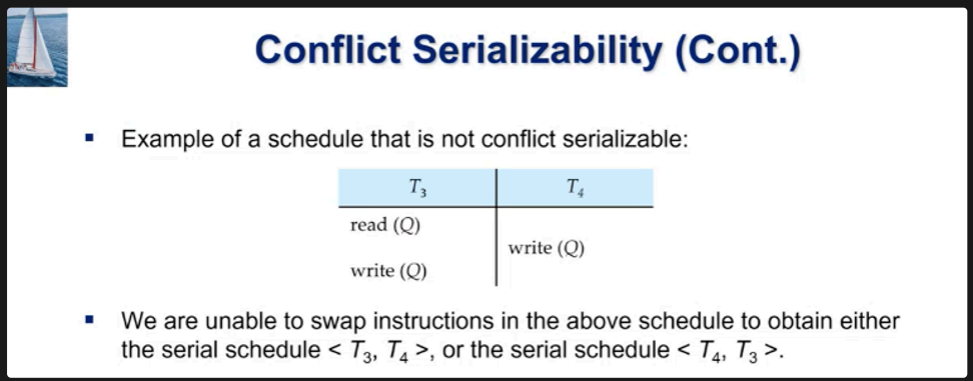


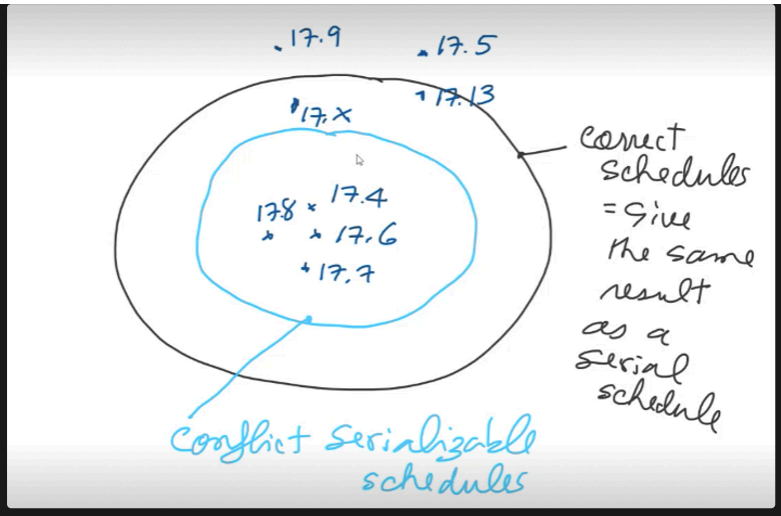
- if Ti, Tj both read Q not matter → before, after don't matter
- We said that I and J are conflict if they are operations by different transaction on the same data item, and **at least one of these instructions is a write operation**

> Our DBMS nowadays follow this, but when it do → it doesn't swap operations → it must have some algorithm to garuntee conflict serilzable

Conflict serializable is an accepted definition when we said we want correctness (not the best)

👉 Example of a schedule that is not conflict serailizable.





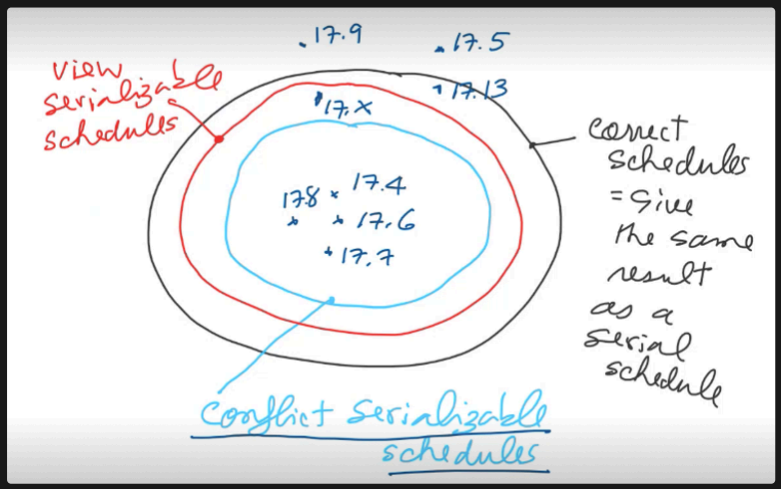
- incorrect schedule won't be allow to run, 17.x and 17.13 will also not allowed to run, if you set level of consistency to be serializable.
- Researcher try to move the blue circle as close to the black one as possible
- how to garuntee the black one?
  - need to be able to access everything, not just sql statement. (if 1 day, your scheduler have access to source code → acheived)
  - can't just determine from sql
- Now, we just use the blue one.

> However, the scheduler not try to swap thing like that, it will follow algorithm that in term garuntee conflict serailizabiliyt(discus in chpter 18)

there also another starndard

✅ View Serializable Schedules

if it view equivalent to serial schedule



View Serializability

- Let *S* and *S'* be two schedules with the same set of transactions. *S* and *S'* are **view equivalent** if the following three conditions are met, for each data item *Q*,
  1. If in schedule S, transaction T_i reads the initial value of *Q*, then in schedule *S'* also transaction T_i must read the initial value of *Q*.
  2. If in schedule S transaction T_i executes **read**(*Q*), and that value was produced by transaction T_j (if any), then in schedule *S'* also transaction T_i must read the value of *Q* that was produced by the same **write**(Q) operation of transaction T_j .
  3. The transaction (if any) that performs the final **write**(*Q*) operation in schedule *S* must also perform the final **write**(*Q*) operation in schedule *S'*.
- As can be seen, view equivalence is also based purely on **reads** and **writes** alone.

- if above 3 conditions are met, S and S′ are view equivalent (view equivalent conditions)

- A schedule *S* is **view serializable** if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but *not* conflict serializable.

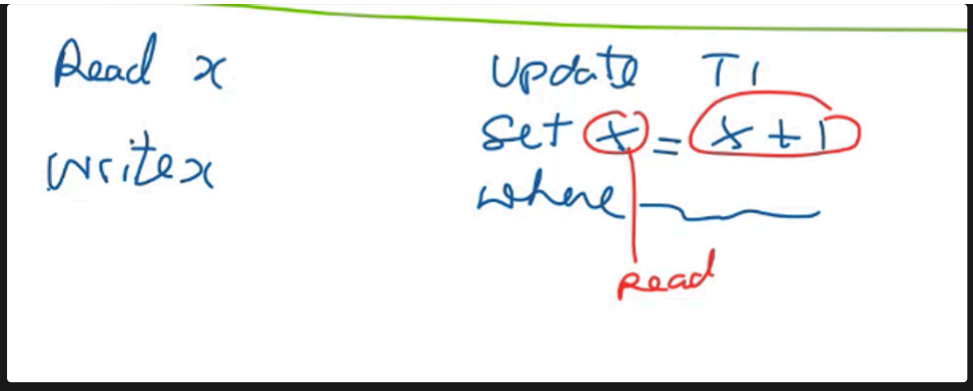
T_{27}	T_{28}	T_{29}
read (*Q*)		
	write (*Q*)	
write (*Q*)		
		write (*Q*)

- What serial schedule is above equivalent to?
- Every view serializable schedule that is not conflict serializable has **blind writes**.

Arrange T27, 28, 29 → give the same result as running this?

- read and then write (update statement)
- read, then modify, then update
- T28, T29 → write Q without read? → blind writes
- view serializable schedule allows you to have blind write, but conflict serializable schedule don't allow

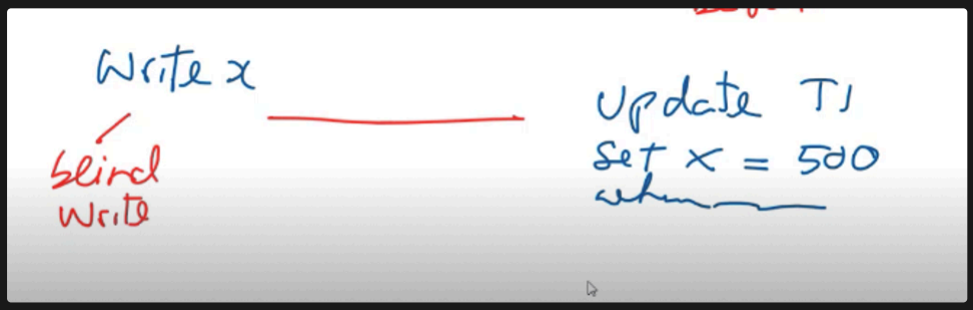
Normal write



when you see read x write x, it kindof like you have to get x data (read) first, in order to update(write). Read old values before writing

- read and then write
- read first, write later
- refer to previous x

Blind write



- T28, 29 → write (Q) without pre read

Example

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 3

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

Schedule 6

- Let check if these are view equivalent based on 3 conditions
  - rule 1
    - T1 read the initial value of A first in schedule 3, and also read A first in schedule 6 (checkkk)
    - T1 read B first in S(the initial value)(schedule 3), T1 read B first in S′(schedule 6 too)
  - rule 2 (same source)
    - (Schedule 3)T2 read A, and this A produce by T1 in S
    - (schedule 6) T2 read A, and this A produce by T1 in S′
    - T2 read B produced by T1, in both S, and S′
  - rule 3
    - T2 write A last in S, T2 also write A last in S′
    - T2 write B last in S, T2 also write B last in S′ too

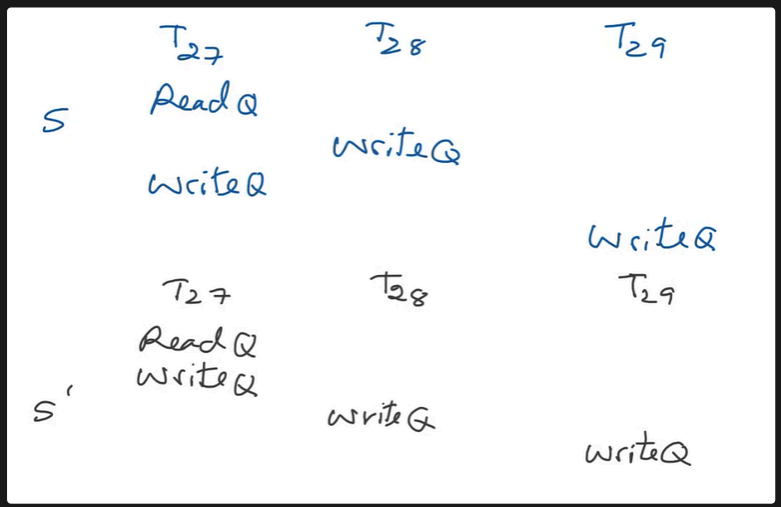
This 2 are view equivalent

and becasue scedule 6 is serial → schedule 3 is view serializabel schdule.

- A view serializable schedule s is a schedule which is view quivalent(3 conditions) to a serial schedule.
  - 3 conditions of view equivalent

Example (17.x)

Are S and S′ view equivalent



Are they view equivalent and why?

- S′ is serial schedule, so in fact, we checking is s a view serialize
- rule 1:
  - checkkk, both read initial value of Q

- rule 2:
  - same source → no read data, only write (free)
- rule 3:
  - checkkk, write final Q in both S and S′

> In summary, conflict serializable is more practical, view serializablility need certain condition to be check and it hard to check? and the benefit is simply→ blind write

Every view serailizable schedule that is not conflict serailizable has blind writes

Most dbms nowadays refer to conflict serialize schedule not the view one

- conflict serialiable → easier to implement

> 👉 for levels of consistency, serializable either mean conflict serailizable(more common) or view seriazable

There important assumption whenw e consider conflict and view serialibility → which is all transaction succeed, all commit successfully, no failure. but in reality, failure take place, we need to consider it

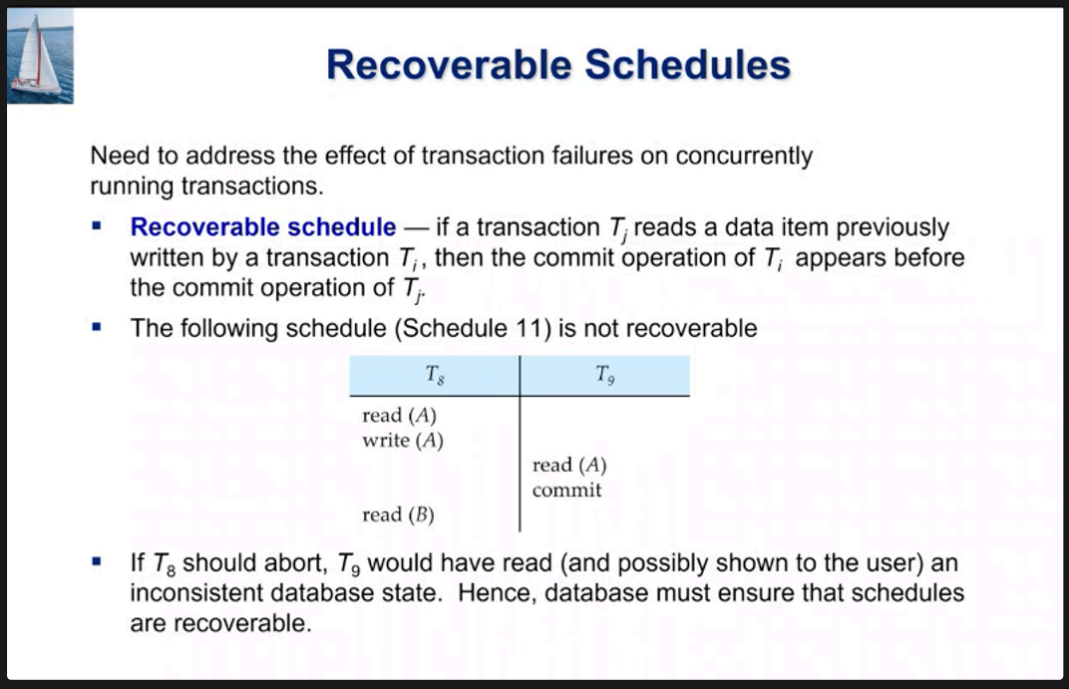
---

✅ Correctness level when we also consider failure

> Important assumption when we consider conflict or view serializaiblity

- the assumption is → all transaction succeed → commit successfully no failure
- but in reality, failure take place, we have to consider failure.

🌱 Recoverable Schedule



- actually, the sources should be commit before the transaction

- the source must commit first before the reader could commit
  - this case reader commit first before the source
- what if T8 eventually failed?
  - it mean T9 read value and commit but the source (T8) rolled back → T9 obtain wrong result (read from rolled back transaction)
  - we don't want this

> This kind of schdule is called a **non-recoverable schedule** because T9 read A and commit, T8 rolled back, we can't rolled back T9. T9 obtain a wrong value

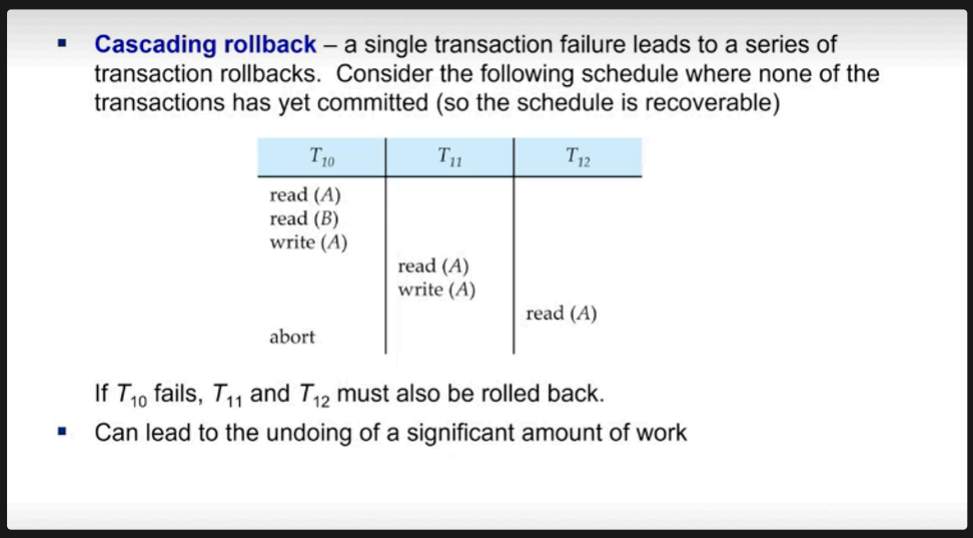
- What we want? - a recoverable schdules

Definition of Recoverable schedule - if a transaction Tj reads a data item previusly written by a transaction Ti, then the commit operation of Ti appears before the commit opreation of Tj

- in fact, recovery schedule is not enough (good condition but still not enough, we need **cascadeless schedule**)
- because we can have kindof recoverable schedule that lead to cascadesless roll back (example below)

🌱 Cascading Rollbacks

This slide show cascading rollback phenomenon (problem)



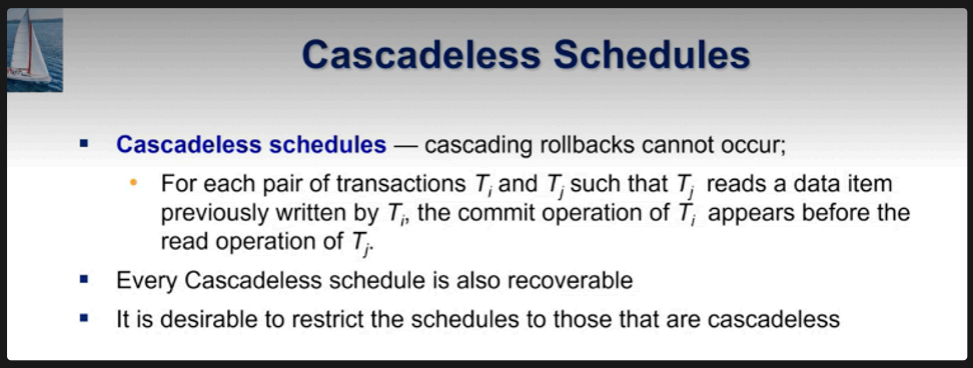
They read in sequence...

- Assume that this is a recoverable schedules → which mean T10 must commit before T11 and then;. before T12 too
- eventually, T10 failed, what happend (abort, rolled back)
- because T10 rolled back, T11 and T12 must also be rolled back
  - waste money and time —> undoing lot of work
- we don't really want this, we don't want this kind of cascading rollbakc

we want casecadeless schedules

- in casecadeless schedules → casecsding rollback can't occur

(default in DBMS nowadays)(every casecadeless is recoverable schedule)

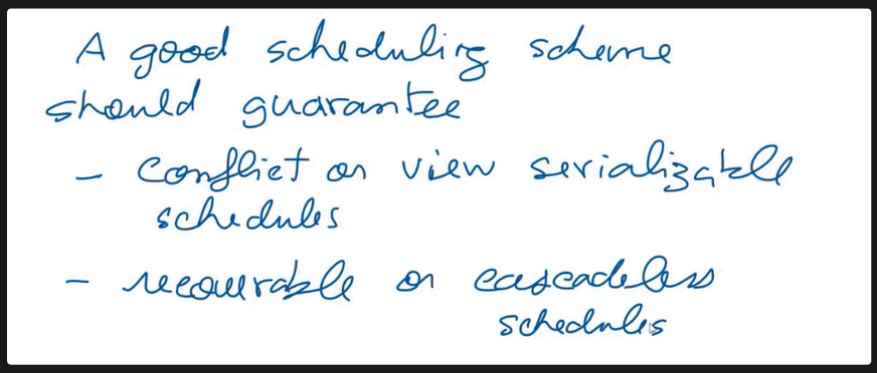


- the source must commit first before the reader can read.
- casecadeless must be recoverabled
  - similar concept to recoverable
    - recoverable → source commit before reader commit
    - casecadeless → source commit first before reader could read
      - strict version of recoverable

- the value must commit before Tj could read

Standard for us for scheduling

What is good scheduling scheme?



A good scheduling scheme should gauruntee conflict or view serializable, this mean when user, programmer submit transaction, the scheudler of DBMS should garuntee to have correct result

- and correct mean confict or view serializable schedules.

as mention earlier, view serialzable is more generous but more complicate

- so in practice, we use **conflict serializable schedules as a standard.**

Also when we consider failure, we need to have recoverable or cascadeless schedules.

> 👉 A good scheduling scheme should gaurantee → conflict or view serializable schedules/ → recoverable or casecadeless schedules.

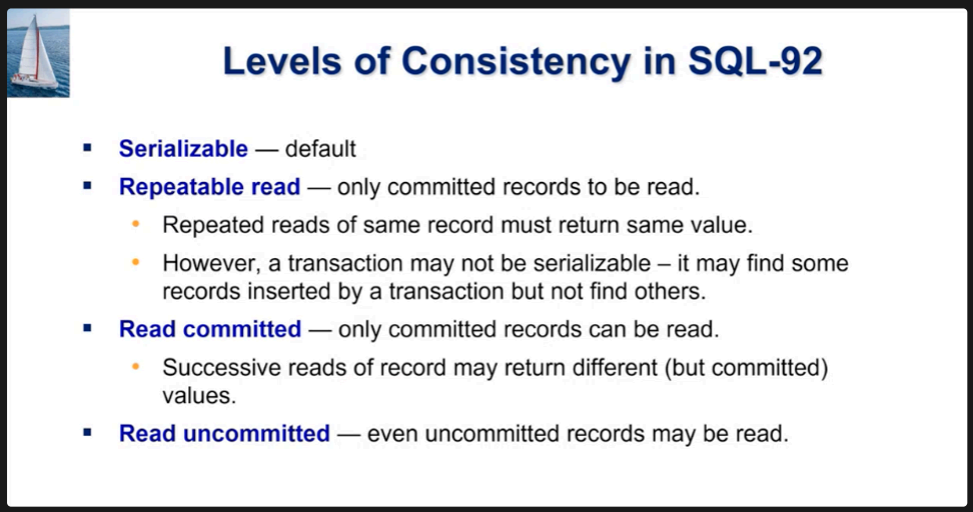
Who implement this/ ensure this good scheduling scheme? People or DBMS

- actually, it possible for human programmer to do all this, but doesn't make sense. Should be DBMS, system software
- in the past, it the job of human programmer. We all know that writing program to meet user requirement is hard enough, so all these thing related topic should be handle by DBMS. → luckily, now, dbms can handle this.
- we'll study later on how to work with this if one day we work without DBMS.
  - if one day we have build scheduler of DBMS by ourself.

Way to say what we want

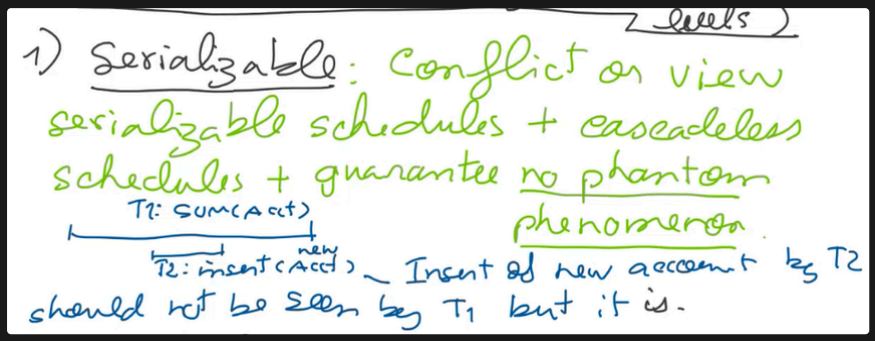
- the correctness we want (way to say) → level of consistency(isolation level)
  - lower degree of consistency useful for gathering approximate information about the database. and therefore reduce the workload of DBMS
  - high level of correctness → more overhead of scheduling, DBMS work hard.

Levels of consistency (Isolation levels)

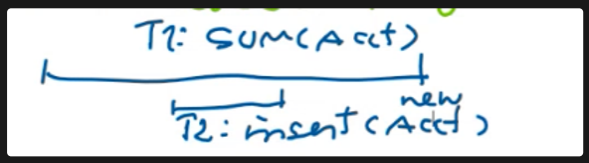


- Different of level of consistency → the result of the same program may not be the same.
- serializable is a default, academically, but not practically
- if not set → use default (mostly, read committed)

1) Serializable - conflict or view serializable schedules (garuntee by DBMS, there are several algorithm to garuntee this) + cascadeless schedule (the source must commit first before the reader can read)(casecadeless also recoverable) + garuntee **no phantom phenamenon**(insert will not get in)



- cascadeless schedule also recoverable scedule
- what is phamtom phenomenon?



  - you have transaction T1 doing sum(ACCT)
    - t2 insert new account
  - Insert of new account by T2 should not eb seen by T1 but it is.
  - the newly insert account and the sum won't conflict → different data item → **weak point of conflict serailizability** (sum of T1 may include the newly inserted value → should not be a case, because It should be like T1 follow by T2) Insert of new account T2 should not be seen by T1 but it is.
    - conflict serializability not consider insertion
    - which should not be a case → because serailiable is T1first follow by T2
  - Weak point of conflict serailiable
- gaurantee no phantom phenomenon, yayy (no insert could get in)

2) Repeatable read (allow insertion during read)(level lower than serializable)

- only commited record to be read (casecadeless schedule)
  - when you want to read, you can't read uncommitted data → this garuntee cascadeless schedule(source must commit first before the reader can read)
- Repeated reads of same record must return same value
  - it mean this is conflict serializable → you don't see value write by someone younger.
- However, transaction may not be serializable - it may find some records inserted by a transaction but not find others.
  - phantom phenomanon can still take place (it may find some record inserted)
- almost the same as serializable, but the phantom phenomenon may take place.

3) Read Committed

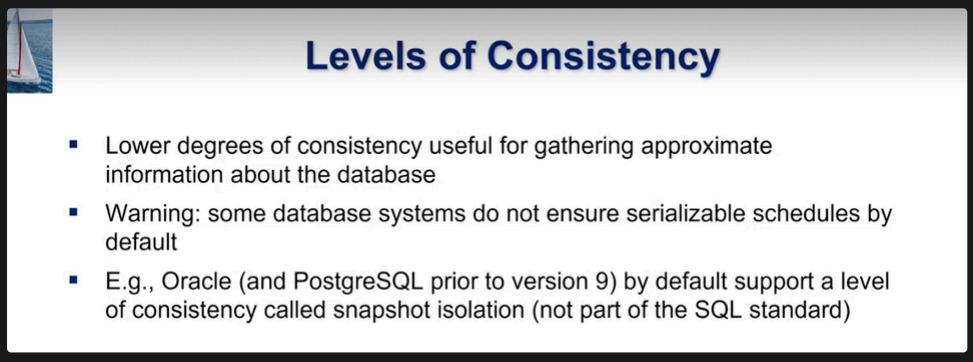
- garuntee cascadeless schedule
- only committed records can be read.
- Successive reads of record may return different (but committed) value.
- (concentrate on read only commit record, not garuntee conflict or view serializability.) - casecadeless schedule

4) Read uncommited (Dirty read)

- even uncommitted record may be read.
- casecadeless schedules are not garunteed.

all this don't allow dirty write → overwrite value that update but haven't commit yet

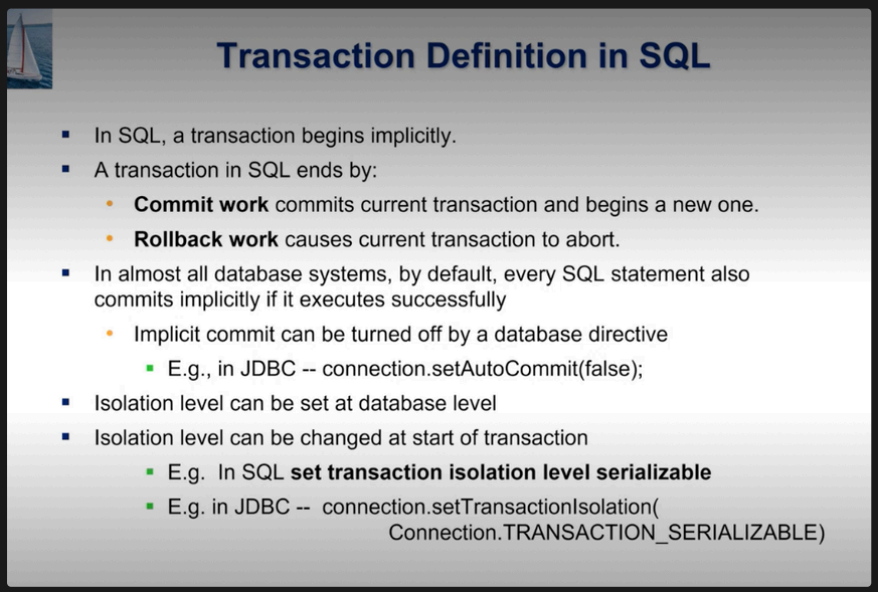
Different setting give different result



- Most vendor set default to be read-committed, because they know customer only concern about the response time, they don't care about correctness
  - nowadays, some use level of consistency for performance tuning.
- if want nice performance, relax correctness.

Nowadays, just tell DBMS you want what level of consistency. (don't have to use any technique)

Setting



✅ Problem that prevent correctness(concurrency control based on problem concept)

(if we can solve problem, we can get correctness)

The 4 concurrency control problem

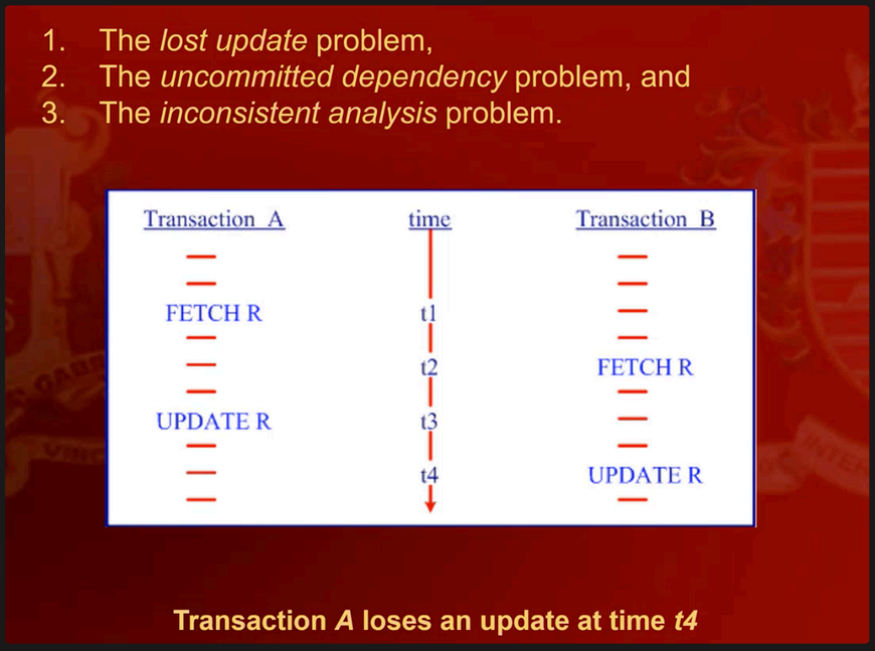
- they are the problem that prevent correct schedule.
- if they are solved → we get correct schedules

Originally, they were 3 of them

1. The lost update problem
2. The uncommitted dependency problem
3. The inconsistent analysis problem
4. (now we have) phantom phenomenon

1. **The lost update problem (not conflict serializable)**

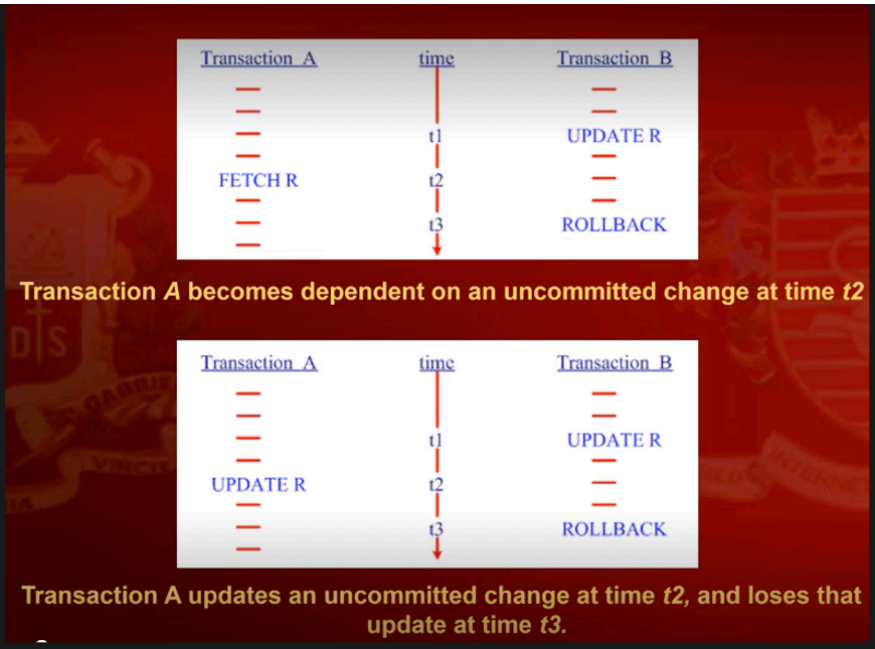
- you lost value as written by A, B overwrite it → work done by A is lost.



- is this conflict serializable? no, because you can't transform this into serial schedule (fetch r and update r conflict)
  - if you set your system to be serializable, this one won't come out.
- B read old value of R, it suppose to read after update R in transaction A.
  - it like running B alone, the work that done by A is lose.

If we use X lock, both X lock R, read R, unlock R → still wrong, doesn't help

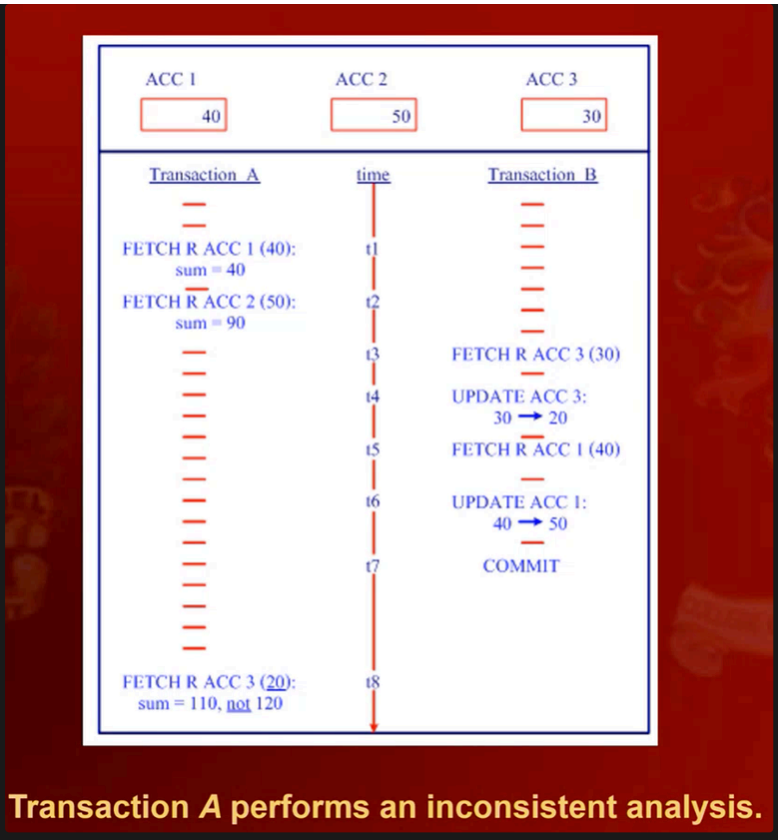
2. **The uncommitted dependency problem**



- First
  - not cascadeless schedule
  - Dirty read
- Second
  - Dirty write

3. **The inconsistent Analysis (not conflict serializable, and incorrect result)**

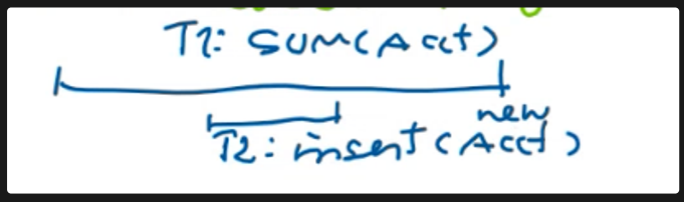
- read only transaction compute read from the data, and some other transaction update the data
  - phantom phenomena is the special case of inconsistent analysis



- is this conflict serializable schedule? no, because you have read(r, acc3) and update (r, acc3) → can't swap (conflict)
- does it give correct result? no

(4) Phantom phenomenon

- you have T1 and T2 insert new account



- when we talk about conflict serailizable, write(A) write(b) not conflict → work on different data item.
  - newly insert account and the sum won't conflict and could get in

👉 Solution

- lot of people don't care about this topic here → because they said they have locking
  - in database, we don't lock process, we lock resources
- like if you working on row A, you want to read row A, you can lock row A using read lock → some body also can read the same row, but other process can't